

feynsage

*Feynman integrals in SageMath,
the lecture in code and a real prediction*

Kazi Abu Rousan

School of Physical Sciences, NISER, Bhubaneswar

Source code and install instructions at github.com/aburousan/feynsage



Drawing made by me

WHAT THIS TUTORIAL IS

It teaches the package feynsage by doing with it the calculations of Prof. B. Ananthanarayan's two-part lecture "An Introduction to Feynman Integrals" (NISER, 2026). These are the tadpole, integration by parts, the bubble and its differential equation, Feynman parameters and the graph polynomials $\mathcal{U}$ and $\mathcal{F}$. Each function is explained in a violet box the first time it is used. The box says what the function takes, what it gives back and what it saves you from doing by hand. The last chapter puts the pieces together for a real prediction, the rate at which the neutral pion decays into two photons.

Every grey box marked **In [n]** is real code. It was run, in this order, in one SageMath session (as in a Jupyter notebook) and the box under it marked **Out [n]** is what came out. Nothing in the output boxes was typed by hand. I am also writing up the lecture itself in full (my notes *Feynman integrals at one loop*). They are not finished yet and will be shared once they are complete.

Contents

1 Getting feynsage	3
1.1 What you need	3
1.2 Download and install	3
1.3 Check and start	3
2 A short tour of feynsage	3
3 Part 1: the tadpole	4
3.1 The solid angle	4
3.2 The tadpole with n powers	4
3.3 The pole and the $\lambda\varphi^4$ tadpole	5
4 Part 1: integration by parts for the tadpole	6
4.1 Integral families	6
4.2 The IBP identity	6
4.3 Solving the identities: reduction	7
4.4 The difference equation and its solution	8
5 Part 1: the basis of one-loop integrals	8
6 Part 1: the bubble	9
6.1 IBP for the bubble	9
6.2 The differential equation	10
6.3 Solving the differential equation	11
6.4 The hypergeometric function in Sage	13
7 Part 1: the fish diagram and Feynman parameters	13
7.1 Ramond's expansion around $D = 4$	14
7.2 Going to Minkowski space: the imaginary part	14
8 Part 2: graph polynomials	15
8.1 The bubble with two masses	16
8.2 The box (Smirnov, Fig. 3.6)	16
8.3 Smirnov's two-loop example	17
8.4 From $\mathcal{U}$ and $\mathcal{F}$ back to the integral	18
8.5 Sir's notebook: completing the square	19
9 A real prediction: the decay rate of the neutral pion	20
9.1 Step 1: the graph, its trees and its polynomials	20
9.2 Step 2: the integral from $\mathcal{U}$ and $\mathcal{F}$	22
9.3 Step 3: the Dirac trace	23
9.4 Step 4: the amplitude and why the quark mass drops out	24
9.5 Step 5: the decay rate	24
10 From a model to a cross section: diagrams like FeynArts	26
10.1 Topologies	26
10.2 Putting fields on the lines	26
10.3 The squared amplitude	26
10.4 The cross section, with the width of the Z	27
11 Processes in one line	28
12 Decays of the Higgs boson: Peskin's Final Project III	30
12.1 (a) and (b): the decays at tree level	30
12.2 (c): $h \rightarrow gg$ at one loop	31
12.3 (e) and (f): $h \rightarrow \gamma\gamma$, quarks and the W loop	31
12.4 (d): Higgs production in pp collisions	33
12.5 (g) and (h): the form factor, $\Gamma(h \rightarrow gg)$ and the branching fractions	33
13 Two problems of Chapter 21: the muon $g - 2$ and $e^+e^- \rightarrow W^+W^-$	35
13.1 Problem 21.1: weak contributions to the muon $g - 2$	35
13.2 Problem 21.2: $e^+e^- \rightarrow W^+W^-$ with longitudinal W bosons	36
13.3 All helicities, and Fig. 21.10 of the book	38
14 Where to go next	42

1 Getting feynsage

1.1 What you need

- **SageMath** 10.7 or newer. It is free. The install script below finds it or installs it.
- **FORM** for the Dirac traces of the last chapter. The script installs it too.
- macOS (Apple silicon or Intel), Linux, or Windows through WSL. SageMath does not run on Windows itself.

1.2 Download and install

The code is at github.com/aburousan/feynsage. Either clone it with git or, on the web page, press **Code** and then **Download ZIP** and unpack it. Then, inside the folder:

```
git clone https://github.com/aburousan/feynsage.git
cd feynsage
./install.sh          # finds or installs SageMath and FORM, then installs feynsage
./install.sh --test   # the same, then runs the tests
```

If SageMath is already there, `sage -pip install --user --no-deps .` does the last step by hand.

1.3 Check and start

```
sage -c "import feynsage; print(feynsage.__file__)" # where it was installed
sage -n jupyterlab                               # notebooks, choose the SageMath kernel
```

Every cell of this tutorial can be run again with `sage docs/tutorial/run_cells.sage`.

2 A short tour of feynsage

feynsage is a Python package that runs inside SageMath, so all of Sage (symbols, exact fractions, calculus, plots) is available next to it. Load it once:

```
In [1]: from feynsage import *
        from feynsage.one_loop import tadpole, bubble_equal_mass, expand_eps, D
```

The first line loads everything that is used most. The second loads a few closed formulas from the module `feynsage.one_loop`. The functions used in this tutorial fall into five groups:

task	functions	lecture
closed formulas	tadpole, bubble_equal_mass, bubble_massless, expand_eps	tadpole, bubble, the ε expansion
families and IBP	family, .ibp, ibp_reduce, .draw, .UF	IBP identities, reduction, completing the square
one-loop functions	A0, B0, C0, D0, finite_part, quick_plot	the basis A, B, C, D
graphs	graph, .U, .F, .spanning_trees, .two_forests, .plot, draw_panels	1-trees, 2-trees, $\mathcal{U}$, $\mathcal{F}$
Dirac traces	momenta, lorentz_indices, gamma, slash, gamma5, dirac_trace, contract, epsilon, dot (written as in a QFT book, done by FORM)	the pion (last chapter)

These are only the functions this tutorial needs. feynsage has many more that the lecture did not reach. Some of them are one-loop tensor integrals in the style of Package-X (`loop`, `PVB`, `PVC`, `PVD`), closed forms of C_0 and D_0 in dilogarithms (`explicit`, `c0_closed`, `d0_closed`), the same closed forms with symbolic invariants and masses (`c0_expand`, `d0_expand`), products of two fermion lines (`line_product`), many integrals at once on all cores (`c0_values`, `d0_values`, `loop_many`, `scan`, `pmap`), infrared poles (the module `ir`), the finite-field reducer `reduce_ff`, symmetries, `diagram` (ready-made graphs like "kite") and pictures of sectors (`draw_sectors`). All

of them are listed with their arguments in [docs/REFERENCE.md](#) in the repository and the notebook [examples/feynsage_walkthrough.ipynb](#) uses most of them. `info(f)` explains any one of them.

Two normalisations appear and each is used in its own place.

- The closed formulas and the families use the Euclidean measure of the lecture with the $\pi^{D/2}$ taken out, $\int d^D k / \pi^{D/2}$ with propagators $k^2 + m^2$.
- `A0`, `B0`, `C0`, `D0` follow Package-X (Minkowski space, $\mu^{2\varepsilon} e^{\varepsilon\gamma_E} \int d^D k / (i\pi^{D/2})$), so that their numbers can be compared with Package-X and LoopTools.

Every feynsage function explains itself. `info(f)` prints what it computes and what its output means and most functions also take `explain=True`. Sage's own functions are explained with `f?` in a notebook.

3 Part 1: the tadpole

The lecture started from the simplest loop integral, the tadpole, in Euclidean signature,

$$I_1 = \int \frac{d^D k}{k^2 + m^2}$$

In $D = 4$ it diverges at large k (ultraviolet). In dimensional regularisation $D = 2\omega$ (Ramond) or $D = 4 - 2\varepsilon$ and the angular integral gives the solid angle of D dimensions.

3.1 The solid angle

From the Gaussian integral, $\Omega_n = 2\pi^{n/2}/\Gamma(n/2)$. Sage handles such formulas directly:

SAGE FUNCTION `var('n')`, `expr.subs(n=4)`

`var` makes a symbol (here n). Formulas built from symbols stay exact. `subs` puts in values. A list in square brackets evaluates the formula several times.

For $n = 2, 3, 4$ it must give the circle, the sphere and the 3-sphere:

```
In [2]: n = var('n')
Omega = 2*pi^(n/2)/gamma(n/2) # from the Gaussian integral
[Omega.subs(n=k) for k in [2, 3, 4]]
```

```
Out[2]: [2*pi, 4*pi, 2*pi^2]
```

3.2 The tadpole with n powers

With $k = m \tan t$ the radial integral is a Beta function and

$$I_n = \int \frac{d^D k}{(k^2 + m^2)^n} = \frac{\pi^\omega \Gamma(n - \omega)}{\Gamma(n) (m^2)^{n-\omega}}$$

FEYNSAGE FUNCTION `tadpole(n, m2)`

The one-loop tadpole $\int d^D k / \pi^{D/2} (k^2 + m^2)^{-n}$ in closed form, $\Gamma(n - D/2) (m^2)^{D/2-n} / \Gamma(n)$.

- `n`: the power of the propagator (a number or a symbol)
- `m2`: the mass squared
- returns a Sage expression in the symbol `D` (the dimension). The $\pi^{D/2}$ is taken out of the measure, so the lecture's I_n is this times π^ω .

```
In [3]: m2 = var('m2')
tadpole(n, m2) # Int d^Dk/pi^(D/2) 1/(k^2 + m^2)^n
```

```
Out[3]: m2^(1/2 D - n) Gamma(-1/2 D + n) / Gamma(n)
```

The lecture also showed that I_n follows from I_1 by differentiating with respect to m^2 ,

$$I_n(m^2) = \frac{(-1)^{n-1}}{(n-1)!} \left(\frac{d}{dm^2} \right)^{n-1} I_1(m^2)$$

SAGE FUNCTION `diff(f, x, k), f.simplify_full()`

`diff` differentiates k times with respect to x . `simplify_full` brings an expression to a simple form, so a difference that is zero prints as 0.

```
In [4]: for k in [2, 3, 4]:
        lhs = tadpole(k, m2)
        rhs = (-1)^(k-1)/factorial(k-1) * diff(tadpole(1, m2), m2, k-1)
        print("n = %d: I_n - (-1)^(n-1)/(n-1)! (d/dm^2)^(n-1) I_1 =" % k,
              (lhs - rhs).simplify_full())

n = 2: I_n - (-1)^(n-1)/(n-1)! (d/dm^2)^(n-1) I_1 = 0
n = 3: I_n - (-1)^(n-1)/(n-1)! (d/dm^2)^(n-1) I_1 = 0
n = 4: I_n - (-1)^(n-1)/(n-1)! (d/dm^2)^(n-1) I_1 = 0
```

3.3 The pole and the $\lambda\varphi^4$ tadpole

Near $D = 4$ the Gamma function has a pole.

FEYNSAGE FUNCTION `expand_eps(expr, order=0, loops=1)`

Puts $D = 4 - 2\epsilon$ into an expression with Gamma functions, multiplies by $e^{L\epsilon\gamma_E}$ ($L = \text{loops}$, the usual convention that removes γ_E) and expands in ϵ up to ϵ^{order} . Gamma functions at negative integers are rewritten first, so the poles come out correctly.

```
In [5]: expand_eps(tadpole(1, m2), 0)
```

```
Out[5]:
```

$$\frac{\epsilon m_2 \log(m_2) - (\epsilon + 1)m_2}{\epsilon}$$

This reads $-m^2/\epsilon - m^2 + m^2 \log m^2$, a simple pole as the lecture said. In $\lambda\varphi^4$ theory (Ramond, Section 4.4, eq. (4.4.3)) the tadpole diagram is $\frac{1}{2}(-\lambda)(\mu^2)^{2-\omega} \int d^{2\omega}\ell / (2\pi)^{2\omega} 1/(\ell^2 + m^2)$. Put in the integral, expand around $\omega = 2$ with $e = 2 - \omega$ and compare with the lecture's result

$$\frac{\lambda m^2}{32\pi^2} \left\{ \frac{1}{2-\omega} + \psi(2) + \ln\left(\frac{4\pi\mu^2}{m^2}\right) + O(2-\omega) \right\}$$

SAGE FUNCTION `expr.series(e, k).truncate()`

The Laurent series of `expr` in `e` up to (not including) e^k , as an ordinary expression.

```
In [6]: lam = var('lam', latex_name=r'\lambda'); mu2 = var('mu2', latex_name=r'\mu^2')
        w, e = var('omega e')
        # (1/2)(-lambda)(mu^2)^(2-w) Int d^(2w)l/(2 pi)^(2w) 1/(l^2 + m^2),
        # the integral done as above
        T = 1/2*(-lam)*mu2^(2-w) * pi^w*gamma(1-w)*m2^(w-1)/(2*pi)^(2*w)
        Texp = T.subs({w: 2-e}).series(e, 1).truncate()
        lecture = lam*m2/(32*pi^2)*(1/e + psi(2) + log(4*pi*mu2/m2))
        gap = (Texp - lecture).canonicalize_radical().simplify_full()
        print("minus the lecture's result:", gap)
        Texp.collect(e)
```

minus the lecture's result: 0

```
Out[6]:
```

$$\frac{\lambda m_2 \log(\pi)}{32\pi^2} - \frac{((\gamma_E - 2\log(2) - \log(\mu^2) - 1)m_2 + m_2 \log(m_2))\lambda}{32\pi^2} + \frac{\lambda m_2}{32\pi^2 e}$$

The difference is exactly zero. The digamma function at 2 is $\psi(2) = 1 - \gamma_E$:

```
In [7]: psi(2), psi(2).n()
```

```
Out[7]: | (-gamma_E + 1, 0.422784335098467)
```

4 Part 1: integration by parts for the tadpole

4.1 Integral families

An **integral family** is a list of propagators $D_1, D_2, \dots$ together with all integrals $\int 1/(D_1^{a_1} D_2^{a_2} \dots)$ with integer powers. IBP relates its members. For the tadpole there is one propagator and the members are $T(n) = \int 1/(k^2 + m^2)^n$.

FEYNSAGE FUNCTION `family(props, kin=None, loops=None, euclidean=False, name="F")`

Builds an integral family from strings.

- `props`: the propagators, each a momentum "k" or ("momentum", "mass") for a massive line
- `kin`: the scalar products of the external momenta, for example {"p^2": "s"}
- `loops`: the loop momenta (guessed from the names if left out)
- `name`: the letter used for its members, "F" if left out. Here "T" for the tadpoles and later "J" for the bubbles, as in the lecture. It is only a label.
- `euclidean=True`: propagators $q^2 + m^2$ as in the lecture (default Minkowski, $q^2 - m^2$)
- returns a `Family`. A member is written with the family's name and the powers of the propagators. For example $T(2)$ is $\int d^D k / \pi^{D/2} 1/(k^2 + m^2)^2$ and for a family with two lines $J(2,1)$ is $\int 1/(D_1^2 D_2)$. A power 0 means the line is absent and a negative power is a numerator. (This F or T or J has nothing to do with the hypergeometric function.) `.info()` prints the family, `.ibp(a)` gives the IBP identities, `.UF()` the graph polynomials.

```
In [8]: tad = family(["k", "m"], euclidean=True, name="T")
        tad.info()
```

```
Integral family 'T'
  loop momenta: k   external momenta: none   Euclidean metric
  D1 = (k)^2 + m^2
  T(a1,...,at) = Int prod_r d^d l_r prod_i D_i^(-a_i), a_i <= 0 is a numerator.
```

4.2 The IBP identity

This is Gauss's law in D dimensions. The integral of a total derivative vanishes,

$$\int d^D k \frac{\partial}{\partial k^\mu} \frac{k^\mu}{(k^2 + m^2)^n} = 0 \Rightarrow 2(\omega - n)I_n + 2nm^2 I_{n+1} = 0$$

FEYNSAGE FUNCTION `fam.ibp(a)`

All IBP identities with the **seed** `a` (a tuple of powers, here $(n,)$ for $T(n)$). For every loop momentum ℓ and every vector v (loop or external) it gives the identity from $\int \partial/\partial \ell^\mu (v^\mu \text{integrand}) = 0$, worked out exactly as in the lecture (derivative, then every scalar product written through the propagators). It returns a list of dictionaries `{powers: coefficient}`. For example `{(1,): d - 2, (2,): 2*m^2}` means $(d - 2)T(1) + 2m^2 T(2) = 0$. `d` is the dimension.

```
In [9]: for k in [1, 2, 3]:
        print("seed T(%d):" % k, tad.ibp((k,)))           # {(n,): c} means c T(n)

seed T(1): [{(1,): d - 2, (2,): 2*m^2}]
seed T(2): [{(2,): d - 4, (3,): 4*m^2}]
seed T(3): [{(3,): d - 6, (4,): 6*m^2}]
```

The first line reads $(d - 2)T(1) + 2m^2 T(2) = 0$, the identity of the lecture with $n = 1$. In general $(d - 2n)T(n) + 2nm^2 T(n + 1) = 0$.

4.3 Solving the identities: reduction

Used again and again, the identities bring every $T(n)$ down to $T(1)$. That is what a reduction program does.

FEYNSAGE FUNCTION `ibp_reduce(fam, targets, method="auto")`

Writes every target integral as a combination of **master integrals**, the members that the identities cannot remove.

- **targets**: the integrals to reduce, by their powers, for example `["T(2)", "T(3)"]` or the same as tuples `[(2,), (3,)]`
- it chooses which seeds to use, finds the symmetries of the family, writes all IBP identities and solves them (Laporta's method). `method = "ff"` (finite fields, fast), `"trimmed"` or `"exact"` (exact rational functions). `"auto"` picks one.
- returns a `Reduction`. `r[target]` is the dictionary `{master: coefficient}`, `r.masters` the list of masters, `r.info()` a summary. In a notebook it prints as typeset equations.

```
In [10]: r = ibp_reduce(tad, ["T(2)", "T(3)", "T(4)"])
         r
```

Out [10]:

$$\begin{aligned} T(2) &= -\frac{(d-2)}{2m^2} T(1) \\ T(3) &= \frac{(d-4)(d-2)}{8m^4} T(1) \\ T(4) &= -\frac{(d-6)(d-4)(d-2)}{48m^6} T(1) \end{aligned}$$

The letters can also be drawn as pictures. `draw` puts the graph of each integral in place of its name. A dot on a line is one extra power of that propagator, so $T(2)$ is the tadpole with one dot.

FEYNSAGE FUNCTION `r.draw(graph, targets=None, rename=None, size=1.5)`

Draws the reduction `r` as equations between Feynman diagrams, one row for each target.

- **graph**: the diagram of the family, made with `graph(...)` (Part 2 explains it). Its lines must be in the same order as the propagators of the family.
- **targets**: which rows to draw (all of them by default)
- **rename**: a dictionary to print a variable under another name, for example `{"kk": "k^2"}`
- a line with power 0 is contracted to a point, so the bubble with one line removed is drawn as a tadpole

```
In [11]: fig = r.draw(graph("A-A:m", {}), size=1.0) # the tadpole: one line from A back to A
```

Out [11]:

$$\begin{aligned} \text{tadpole with 1 dot} &= \left(-\frac{d-2}{2m^2} \right) \text{tadpole with 0 dots} \\ \text{tadpole with 2 dots} &= \left(\frac{(d-2)(d-4)}{8m^4} \right) \text{tadpole with 0 dots} \\ \text{tadpole with 3 dots} &= \left(-\frac{(d-2)(d-4)(d-6)}{48m^6} \right) \text{tadpole with 0 dots} \end{aligned}$$

The ratios must be exactly those of the Gamma functions above. The coefficients come back as rational functions of d . `SR(str(...))` turns one into an ordinary Sage expression so that it can be compared with `tadpole`:

```
In [12]: mm = var('m')
         for k in [2, 3, 4]:
             c = SR(str(r[(k,)][(1,)])) .subs({SR.var('d'): D})
             print("T(%d)/T(1) from IBP equals the Gamma-function ratio:" % k,
                   bool((c - tadpole(k, mm^2)/tadpole(1, mm^2)).simplify_full() == 0))

T(2)/T(1) from IBP equals the Gamma-function ratio: True
T(3)/T(1) from IBP equals the Gamma-function ratio: True
T(4)/T(1) from IBP equals the Gamma-function ratio: True
```

4.4 The difference equation and its solution

Argeri and Mastrolia write the identity for $U(n) = \pi^{-D/2} \int d^D k / (k^2 + 1)^n$ as a difference equation, $-(n - D/2)U(n) + nU(n+1) = 0$. Mathematica's `RSolve` solves it once $U(1)$ is given. The solution is $U(n) = \Gamma(n - D/2) / (\Gamma(1 - D/2)\Gamma(n)) U(1)$. Put it back:

```
In [13]: Uformula = lambda n: gamma(n - D/2)/(gamma(1 - D/2)*gamma(n)) # times U(1)
         (- (n - D/2)*Uformula(n) + n*Uformula(n + 1)).simplify_full()
```

```
Out[13]: 0
```

The lecture also solved it with a **factorial series**, $U(n) = \int_0^1 dt t^{n-1} v(t)$. Putting this into the difference equation and integrating by parts (with $v(1) = 0$) gives a first-order differential equation for v ,

$$\frac{v'(t)}{v(t)} = \frac{D/2 - t}{t(t-1)} \Rightarrow v(t) = v_0 t^{-D/2} (1-t)^{D/2-1}$$

and the asymptotic estimate fixes $v_0 = 1/\Gamma(D/2)$.

SAGE FUNCTION `desolve(eq, y, ivar=t), numerical_integral(f, a, b)`
`desolve` solves an ordinary differential equation for the function y of t (here $_C$ is the free constant v_0).
`numerical_integral` integrates numerically and returns the value and an error estimate. `[0]` takes the value.

Sage solves the equation and the integral gives back $U(n)$ (here $D = 2.6$, $n = 3$):

```
In [14]: t = var('t')
         v = function('v')(t)
         sol = desolve(diff(v, t) == (D/2 - t)/(t*(t - 1))*v, v, ivar=t)
         print("v(t) =", sol)
         Dv, nv = 2.6, 3
         num = numerical_integral(t^(nv - 1) * t^(-Dv/2)*(1 - t)^(Dv/2 - 1) / gamma(Dv/2), 0, 1)[0]
         print("Int_0^1 t^(n-1) v(t) dt with v0 = 1/Gamma(D/2):", num)
         print("U(n) = Gamma(n - D/2)/Gamma(n):", tadpole(nv, 1).subs(D=Dv).n())

v(t) = _C*e^(1/2*(D - 2)*log(t - 1) - 1/2*D*log(t))
Int_0^1 t^(n-1) v(t) dt with v0 = 1/Gamma(D/2): 0.4543193697876147
U(n) = Gamma(n - D/2)/Gamma(n): 0.454319366426645
```

5 Part 1: the basis of one-loop integrals

Every one-loop integral can be reduced (up to terms of order ε) to four scalar functions, A (tadpole), B (bubble), C (triangle) and D (box). The method is called Passarino-Veltman reduction. Package-X computes them in Mathematica and FeynCalc uses it through FeynHelpers. FIRE and Kira do the IBP reduction for many loops.

FEYNSAGE FUNCTION `A0(m)`, `B0(s, m1, m2)`, `C0(s1, s12, s2, m0, m1, m2)`, `D0(...)`

The four scalar one-loop functions in the Package-X normalisation (Minkowski space). `B0(s, m1, m2)` has propagators (ℓ, m_1) and $(\ell + p, m_2)$ with $p^2 = s$. A_0 and B_0 come back as formulas with the pole $1/\varepsilon$ and the scale μ . A finite C_0 or D_0 stays a symbol until `.n()` asks for its number (more than 30 digits are available).

FEYNSAGE FUNCTION `finite_part(expr, mu_value=None)`, `uv_part(expr)`

The ε^0 part and the coefficient of $1/\varepsilon$ of an expression. `mu_value` sets the scale μ to a number.

```
In [15]: print("A0(1)                =", A0(1))
print("B0(5; 1, 1), finite part =", finite_part(B0(5, 1, 1), 1).n(digits=12))
print("C0(1, 2, 3; 1, 2, 3)      =", C0(1, 2, 3, 1, 2, 3).n(digits=12))
print("D0(1,2,3,4; -5,-6; 1,...) =", D0(1, 2, 3, 4, -5, -6, 1, 1, 1, 1).n(digits=12))
```

```
A0(1)                = 1/eps + log(mu^2) + 1
B0(5; 1, 1), finite part = 1.56959105904 + 1.40496294621*I
C0(1, 2, 3; 1, 2, 3)      = -0.142398839895
D0(1,2,3,4; -5,-6; 1,...) = 0.272791904368
```

6 Part 1: the bubble

The lecture followed Argeri-Mastrolia (Section 6 of their review) and Ramond, with equal masses, external momentum k and loop momentum p (Argeri and Mastrolia divide the measure by $(2\pi)^{D-2}$, which only changes the overall factor),

$$J_{n_1 n_2} = \int d^D p \frac{1}{D_1^{n_1} D_2^{n_2}}, \quad D_1 = p^2 + m^2, \quad D_2 = (p - k)^2 + m^2$$

In family this is two massive propagators. The name `kk` is just k^2 . A Sage variable name cannot contain $\wedge$, so k^2 is written `kk` (and $k \cdot p$ would be `kp`):

```
In [16]: bub = family([(("p", "m"), ("p - k", "m"))], kin={"k^2": "kk"}, loops=["p"], euclidean=True,
name="J")
bub.info()
```

```
Integral family 'J'
  loop momenta: p   external momenta: k   Euclidean metric
  D1 = (p)^2 + m^2
  D2 = (p - 1*k)^2 + m^2
  J(a1,...,at) = Int prod_r d^d l_r prod_i D_i^(-a_i), a_i <= 0 is a numerator.
```

6.1 IBP for the bubble

Now there are two vectors to put inside the derivative, the internal momentum p and the external one k , so `ibp` gives two identities:

```
In [17]: for ident in bub.ibp((1, 1)):
terms = ["(%s) J(%s)" % (c, ",".join(map(str, a))) for a, c in ident.items()]
print(" + ".join(terms), "= 0")
```

```
(d - 3) J(1,1) + (2*m^2) J(2,1) + (2*m^2 + kk) J(1,2) + (-1) J(0,2) = 0
(-kk) J(2,1) + (1) J(2,0) + (kk) J(1,2) + (-1) J(0,2) = 0
```

A dot on a line means one more power of that propagator. `ibp_reduce` solves the identities and gives the two results of the lecture,

$$J_{21} = -\frac{D-3}{k^2 + 4m^2} J_{11} + \frac{1}{k^2 + 4m^2} T(2), \quad T(2) = -\frac{D-2}{2m^2} T(1)$$

where $J(0,1)$ is the tadpole $T(1)$ (line 1 absent, power 0):

```
In [18]: rb = ibp_reduce(bub, ["J(2,1)", "J(1,2)", "J(0,2)"])
rb
```

```
Out[18]:
```

$$\begin{aligned}
 J(2,1) &= -\frac{(d-3)}{(4m^2+kk)} J(1,1) \\
 &\quad -\frac{(d-2)}{2m^2(4m^2+kk)} J(0,1) \\
 J(1,2) &= -\frac{(d-3)}{(4m^2+kk)} J(1,1) \\
 &\quad -\frac{(d-2)}{2m^2(4m^2+kk)} J(0,1) \\
 J(0,2) &= -\frac{(d-2)}{2m^2} J(0,1)
 \end{aligned}$$

The coefficients are rational functions in the variables d, kk, m . `.parent().gens()` hands these variables to us, so the comparison with the lecture is exact:

```
In [19]: d_, kk_, m_ = rb[(2, 1)][(1, 1)].parent().gens()
print("dotted bubble: ", rb[(2, 1)][(1, 1)] == -(d_ - 3)/(kk_ + 4*m_^2),
      " tadpole part:", rb[(2, 1)][(0, 1)] == 1/(kk_ + 4*m_^2) * (-(d_ - 2)/(2*m_^2)))
print("dotted tadpole: ", rb[(0, 2)][(0, 1)] == -(d_ - 2)/(2*m_^2))

dotted bubble: True  tadpole part: True
dotted tadpole: True
```

The same reduction as pictures. The graph is two lines between the vertices A and B , with k coming in at A and going out at B . `rename` prints kk as k^2 :

```
In [20]: gb = graph("A-B:m, A-B:m", {"A": "k", "B": "-k"}, kin={"k^2": "kk"}, euclidean=True)
fig = rb.draw(gb, targets=[(2, 1), (0, 2)], rename={"kk": "k^2"}, size=1.0)
```

```
Out[20]:
```

$$\begin{aligned}
 \text{Bubble diagram} &= \left(-\frac{d-3}{4m^2+k^2}\right) \text{Bubble diagram} + \left(-\frac{d-2}{2(4m^2+k^2)m^2}\right) \text{Tadpole diagram} \\
 \text{Tadpole diagram} &= \left(-\frac{d-2}{2m^2}\right) \text{Tadpole diagram}
 \end{aligned}$$

6.2 The differential equation

J depends on k only through k^2 , so $k_\mu \partial J / \partial k_\mu = 2k^2 \partial J / \partial k^2$. Differentiating under the integral,

$$k_\mu \frac{\partial J}{\partial k_\mu} = T(2) - J_{11} - k^2 J_{12}$$

The reduction above already contains J_{12} and $T(2)$ in terms of the masters, so the right-hand side becomes $AJ + BT(1)$. That is the differential equation. Compare it with the lecture,

$$\frac{dJ}{dk^2} + \frac{1}{2} \left[\frac{1}{k^2} - \frac{D-3}{k^2+4m^2} \right] J = -\frac{D-2}{4m^2} \left[\frac{1}{k^2} - \frac{1}{k^2+4m^2} \right] T(1)$$

```
In [21]: # k.dJ/dk = J(0,2) - J(1,1) - k^2 J(1,2) and dJ/dk^2 = (k.dJ/dk)/(2 k^2);
# reduce the right side
kk = SR.var('kk'); dd = SR.var('d'); m = SR.var('m')
c = lambda a, mm_: SR(str(rb[a].get(mm_, 0)))
A = (c((0, 2), (1, 1)) - 1 - kk*c((1, 2), (1, 1)))/(2*kk) # coefficient of J(1,1)
B = (c((0, 2), (0, 1)) - kk*c((1, 2), (0, 1)))/(2*kk) # coefficient of T(1)
print("dJ/dk^2 = A J + B T(1) with")
print("  A =", A.factor())
print("  B =", B.factor())
print("as in the notes:",
      bool((A + (1/kk - (dd - 3)/(kk + 4*m^2))/2).simplify_full() == 0),
      bool((B + (dd - 2)/(4*m^2)*(1/kk - 1/(kk + 4*m^2))).simplify_full() == 0))

dJ/dk^2 = A J + B T(1) with
A = 1/2*(d*kk - 4*m^2 - 4*kk)/((4*m^2 + kk)*kk)
B = -(d - 2)/((4*m^2 + kk)*kk)
as in the notes: True True
```

6.3 Solving the differential equation

Now we solve it, with its boundary condition. Put $k^2 = 4m^2x$, so that $dJ/dx = 4m^2 dJ/dk^2$. `partial_fraction` splits the coefficients into simple poles. For finite x the equation is singular only at $x = 0$ ($k^2 = 0$) and $x = -1$ (the threshold $k^2 = -4m^2$ in this Euclidean metric):

```
In [22]: x = var('x')
# write k^2 = 4 m^2 x; then dJ/dx = 4 m^2 dJ/dk^2
Ax = (4*m^2*A).subs(kk=4*m^2*x).subs({dd: D}).simplify_full()
Bx = (4*m^2*B).subs(kk=4*m^2*x).subs({dd: D}).simplify_full()
print("dJ/dx = (%s) J" % Ax, partial_fraction(x))
print("          + (%s) T(1)" % Bx, partial_fraction(x))

dJ/dx = (1/2*(D - 3)/(x + 1) - 1/2/x) J
        + (1/4*(D - 2)/(m^2*(x + 1)) - 1/4*(D - 2)/(m^2*x)) T(1)
```

SAGE FUNCTION `expr.subs(kk=...)`, `expr.subs({dd: D})`, `expr.partial_fraction(x)`
`subs` replaces a variable. The keyword form `subs(kk=...)` works when the Python name and the Sage name are the same. `dd` is the Sage variable printed `d`, so it goes in a dictionary. `partial_fraction(x)` writes a rational function of x as a sum of simple fractions.

The homogeneous equation. `desolve` gives $J_0 = C x^{-1/2} (1+x)^{(D-3)/2}$:

```
In [23]: J0 = function('J0')(x)
hom = desolve(diff(J0, x) == Ax*J0, J0, ivar=x) # the homogeneous equation
hom.canonicalize_radical() # write e^(a log u) as u^a
```

```
Out[23]:
```

$$\frac{C(x+1)^{\frac{1}{2}D-\frac{3}{2}}}{\sqrt{x}}$$

The boundary condition. At $k^2 = 0$ the bubble is $J(D, 0) = \int d^D p / (p^2 + m^2)^2 = T(2)$, a finite number. The homogeneous solution behaves as $x^{-1/2}$ there, so it must not appear. Try a power series $J = \sum_n c_n x^n$, which is regular at $x = 0$. Multiply the equation by $x(1+x)$ to clear the denominators and ask that every power of x cancels. The x^0 equation does not contain any free constant. It **fixes** c_0 and the value is exactly $T(2)$. So regularity at $x = 0$ and the boundary condition are the same statement. Every higher c_n then follows from the one before:

```
In [24]: N = 6
cs = [var('c%d' % i) for i in range(N + 1)]
T1 = var('T1') # the master T(1)
Js = sum(cs[i]*x^i for i in range(N + 1)) # J as a power series in x
eq = (x*(1 + x)*(diff(Js, x) - Ax*Js - Bx*T1)).simplify_rational().expand()
eqs = [eq.coefficient(x, k) == 0 for k in range(N)]
print("the x^0 equation:", eqs[0])
sol = solve(eqs, cs[:N], solution_dict=True)[0]
is_T2 = bool((sol[cs[0]] + (D - 2)/(2*m^2)*T1).simplify_full() == 0)
print("so J(D, 0) = c0 =", sol[cs[0]].factor(), " equal to T(2):", is_T2)
for n in range(4):
    ratio = (sol[cs[n + 1]]/sol[cs[n]]).factor()
    hyp = -(2 - D/2 + n)*(1 + n)/((3/2 + n)*(1 + n)) # term ratio of the 2F1
    same = bool((ratio - hyp).simplify_full() == 0)
    print("c%d/c%d =" % (n + 1, n), ratio, " same as 2F1:", same)

the x^0 equation: 1/2*c0 + 1/4*D*T1/m^2 - 1/2*T1/m^2 == 0
so J(D, 0) = c0 = -1/2*(D - 2)*T1/m^2 equal to T(2): True
c1/c0 = 1/3*D - 4/3 same as 2F1: True
c2/c1 = 1/5*D - 6/5 same as 2F1: True
c3/c2 = 1/7*D - 8/7 same as 2F1: True
c4/c3 = 1/9*D - 10/9 same as 2F1: True
```

SAGE FUNCTION `solve(eqs, vars, solution_dict=True)`, `expr.coefficient(x, k)`
`coefficient(x, k)` takes the coefficient of x^k in an expanded expression. `solve` solves a list of equations for the listed variables. With `solution_dict=True` it returns a list of dictionaries {variable: value}.

The ratio of two neighbouring terms of ${}_2F_1(a, b; c; z) = \sum_n (a)_n (b)_n / ((c)_n n!) z^n$ is $(a + n)(b + n) / ((c + n)(1 + n)) z$. The ratios c_{n+1}/c_n found above are exactly this with $a = 2 - D/2$, $b = 1$, $c = 3/2$, $z = -x$. With $c_0 = T(2) = \Gamma(2 - D/2) m^{D-4}$ the solution is

$$J(D, k^2) = \Gamma(2 - D/2) m^{D-4} {}_2F_1(2 - D/2, 1; 3/2; -k^2/4m^2)$$

FEYNSAGE FUNCTION `bubble_equal_mass(p2, m2)`
 The equal-mass bubble $\int d^D p / \pi^{D/2} 1 / ((p^2 + m^2)((p - k)^2 + m^2))$ at $k^2 = p2$, this solution of the differential equation. The kinematic variable sits in the argument and the dimension in the parameters.

A numerical check. Forget the closed form and integrate the equation with a numerical solver (SciPy's `solve_ivp`), starting next to $x = 0$ from the boundary value. The numbers agree with the hypergeometric function to all ten digits shown ($D = 3.3$, $m = 1$):

```
In [25]: from scipy.integrate import solve_ivp
Dv, mv = 3.3, 1
T1v = float(tadpole(1, mv^2).subs(D=Dv))
fA = fast_callable(Ax.subs(m=mv, D=Dv), vars=[x])
fB = fast_callable(Bx.subs(m=mv, D=Dv), vars=[x])
x0 = 1e-6 # start next to the boundary x = 0
J_start = float(sum(sol[cs[i]].subs(D=Dv, m=mv, T1=T1v)*x0^i for i in range(N)))
num = solve_ivp(lambda t, y: [fA(t)*y[0] + fB(t)*T1v], [x0, 2.0], [J_start],
                rtol=1e-11, atol=1e-13, dense_output=True)
for xv in [0.125, 0.5, 2.0]:
    exact = bubble_equal_mass(4*mv^2*xv, mv^2).subs(D=Dv).n() # Gamma(2-D/2) m^(D-4) 2F1
    print("x = %.3f solved numerically: %.10f closed form: %.10f"
          % (xv, num.sol(xv)[0], exact))

x = 0.125 solved numerically: 2.4765120121 closed form: 2.4765120121
x = 0.500 solved numerically: 2.3095817834 closed form: 2.3095817834
x = 2.000 solved numerically: 1.9299216509 closed form: 1.9299216509
```

SAGE, SCIPY FUNCTION `fast_callable(expr, vars=[x]), solve_ivp(f, [x0, x1], y0, dense_output=True)`
`fast_callable` turns a Sage expression into a fast numerical function. `solve_ivp` integrates $y' = f(x, y)$ from x_0 to x_1 with the starting value y_0 . `num.sol(x)` gives the solution at any x in between.

6.4 The hypergeometric function in Sage

Sage knows ${}_2F_1$ as `hypergeometric([a, b], [c], z)`. `.n(digits=25)` evaluates it to 25 digits (through `mpmath`, which Sage contains). Below the same number is found from the series itself and directly from `mpmath`. The parameters are exact fractions, since a decimal number like `0.35` would limit the precision to 16 digits. For special parameters Maxima can rewrite the function in elementary functions. At $D = 3$ the bubble is an arc tangent:

```
In [26]: a, b, c, z = 7/20, 1, 3/2, -1/2          # 2F1(2 - D/2, 1; 3/2; -x) at D = 3.3, x = 1/2
h = hypergeometric([a, b], [c], z)
print("Sage: ", h.n(digits=25))
poch = rising_factorial                          # (a)_n = a (a+1) ... (a+n-1)
term = lambda n: poch(a, n)*poch(b, n)/poch(c, n)*z^n/factorial(n)
print("the series: ", sum(term(n) for n in range(80)).n(digits=25))
import mpmath; mpmath.mp.dps = 25
print("mpmath.hyp2f1: ", mpmath.hyp2f1(a, b, c, z))
assume(x > 0)
h3 = hypergeometric([1/2, 1], [3/2], -x)          # D = 3
print("D = 3 in closed form:", h3.simplify_hypergeometric(algorithm='maxima'))

Sage:      0.9070889481401101364449382
the series: 0.9070889481401101364449382
mpmath.hyp2f1: 0.9070889481401101364449382
D = 3 in closed form: arctan(sqrt(x))/sqrt(x)
```

SAGE FUNCTION `hypergeometric([a, b], [c], z), rising_factorial(a, n), simplify_hypergeometric(algorithm='maxima')`
`hypergeometric` is the generalised hypergeometric function ${}_pF_q$ with the lists of upper and lower parameters. `rising_factorial(a, n)` is the Pochhammer symbol $(a)_n = a(a+1)\dots(a+n-1)$. `simplify_hypergeometric` looks for a closed form. Maxima needs to know the sign of x , so `assume(x > 0)` comes first.

7 Part 1: the fish diagram and Feynman parameters

The fish diagram of $\lambda\varphi^4$ is the same bubble. The lecture combined the two denominators with the Feynman trick,

$$\frac{1}{AB} = \int_0^1 \frac{dx}{[xA + (1-x)B]^2}$$

SAGE FUNCTION `assume(cond), integrate(f, x, a, b)`
`integrate` does definite integrals exactly. `assume` tells it what it needs to know about the symbols (here that $A, B > 0$).

```
In [27]: A_, B_, x = var('A B x')
assume(A_ > 0, B_ > 0, B_ - A_ > 0)          # any order of A and B gives the same
integrate(1/(x*A_ + (1-x)*B_)^2, x, 0, 1).factor()
```

```
Out [27]: 1
           AB
```

After the shift $\ell \rightarrow \ell + p(1-x)$ the loop integral is a tadpole with mass² $m^2 + p^2x(1-x)$ and two powers, so `tadpole(2, ...)` gives it. Only the integral over the Feynman parameter x is left. Done numerically it equals the hypergeometric result of the differential equation ($D = 3.3$, $m = 1$, $p^2 = 3$):

```
In [28]: Dv, mv, pv = 3.3, 1, 3
# after the shift l -> l + p(1 - x) the loop integral is a tadpole
# with mass^2 = m^2 + p^2 x(1 - x)
tad_x = lambda xx: tadpole(2, mv^2 + pv*xx*(1 - xx)).subs(D=Dv).n()
fp = numerical_integral(tad_x, 0, 1)[0]
print("Feynman parameter integral:", fp)
print("bubble from the DE (2F1): ", bubble_equal_mass(pv, mv^2).subs(D=Dv).n())
```

Feynman parameter integral: 2.2221997148168473
bubble from the DE (2F1): 2.22219971481685

7.1 Ramond's expansion around $D = 4$

Expanding around $\omega = 2$ the finite part is $-\int_0^1 dx \ln(1 + x(1-x)p^2/m^2)$, which Ramond (eqs. (4.4.15) and (4.4.16)) writes as $2 - \sqrt{1 + 4m^2/p^2} \ln[(\sqrt{1 + 4m^2/p^2} + 1)/(\sqrt{1 + 4m^2/p^2} - 1)]$:

```
In [29]: p2v = 3
finite = -numerical_integral(lambda xx: log(1 + xx*(1 - xx)*p2v), 0, 1)[0]
b = sqrt(1 + 4/p2v)
print("finite part, integral over x: ", finite)
print("Ramond: 2 - b ln((b + 1)/(b - 1)):", (2 - b*log((b + 1)/(b - 1))).n())
```

finite part, integral over x: -0.3933253674083756
Ramond: 2 - b ln((b + 1)/(b - 1)): -0.393325367408375

7.2 Going to Minkowski space: the imaginary part

In Minkowski space $p^2 \rightarrow -s$ and above the threshold $s > 4m^2$ the argument of the logarithm turns negative. Then $\ln z = \ln|z| + i\pi$ for $z < 0$ and the bubble gets an imaginary part. By the optical theorem it must, because above threshold the two particles in the loop can be real. $B0$ is the same bubble in Minkowski space, so its imaginary part should be $\pi\beta$ with $\beta = \sqrt{1 - 4m^2/s}$ (here $m = 1$):

```
In [30]: print("ln(-2)      =", log(-2).n())
print("ln 2 + i pi =", (log(2) + I*pi).n())
for sv in [3, 5, 10]:
    val = finite_part(B0(sv, 1, 1), 1).n(digits=12)
    beta = sqrt(1 - 4/sv) if sv > 4 else 0
    print("s = %2d (m = 1):  Im of the bubble = %.10f    pi*beta = %.10f"
          % (sv, imag(val), (pi*beta).n()))
```

ln(-2) = 0.693147180559945 + 3.14159265358979*I
ln 2 + i pi = 0.693147180559945 + 3.14159265358979*I
s = 3 (m = 1): Im of the bubble = 0.0000000000 pi*beta = 0.0000000000
s = 5 (m = 1): Im of the bubble = 1.4049629462 pi*beta = 1.4049629462
s = 10 (m = 1): Im of the bubble = 2.4334672056 pi*beta = 2.4334672056

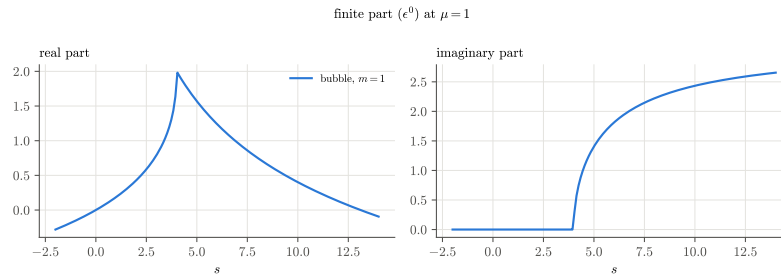
FEYNSAGE FUNCTION `quick_plot`(exprs, (s, a, b), labels=None)

Plots one or more expressions in s from a to b . For an expression with $1/\varepsilon$ it plots the finite part. Complex values are drawn as the real part (full line) and the imaginary part (dashed). Returns a matplotlib figure.

The imaginary part starts at $s = 4$:

```
In [31]: from feynsage.plotting import quick_plot, set_theme
set_theme()
s = var('s')
fig = quick_plot([B0(s, 1, 1)], (s, -2, 14), labels=["bubble, $m = 1$"])
```

Out[31]:



8 Part 2: graph polynomials

The second lecture combined the denominators of any diagram at once and read the result off the graph. Take a diagram with L loops and N propagators $D_j = q_j^2 - m_j^2 + i0$ (Minkowski space), raised to powers ν_j . Its **Feynman integral** is the integral over all loop momenta of the product of propagators. With Feynman parameters it becomes

$$\begin{aligned} I(\nu_1, \dots, \nu_N) &= \int \prod_{l=1}^L \frac{d^D \ell_l}{i\pi^{D/2}} \frac{1}{D_1^{\nu_1} \dots D_N^{\nu_N}} \\ &= (-1)^{N_\nu} \frac{\Gamma(N_\nu - LD/2)}{\prod_j \Gamma(\nu_j)} \int_0^\infty \prod_{j=1}^N dx_j x_j^{\nu_j-1} \delta\left(1 - \sum x_j\right) \frac{\mathcal{U}^{N_\nu - (L+1)D/2}}{\mathcal{F}^{N_\nu - LD/2}} \end{aligned}$$

with $N_\nu = \sum_j \nu_j$. This is not yet the scattering amplitude. The amplitude contains such integrals, multiplied by the couplings and by the numerator from the Dirac algebra (the pion in the last chapter shows how they fit together). The sign $(-1)^{N_\nu}$ comes from the Minkowski propagators. In Euclidean space, with $d^D \ell / \pi^{D/2}$ and propagators $\ell^2 + m^2$, the same formula holds without it.

The two polynomials come from the graph:

- A 1-tree (spanning tree) is what is left after erasing L lines so that the rest touches every vertex without a loop. Each gives the product of the x_i of the erased lines. $\mathcal{U}$ is their sum, homogeneous of degree L .
- A 2-tree is what is left after erasing one more line, so that the graph falls into two pieces. Each gives the product of the erased x_i times the square of the momentum flowing from one piece to the other. Their sum is V and $\mathcal{F} = V + \mathcal{U} \sum_i x_i m_i^2$.

FEYNSAGE FUNCTION `graph(edges, legs, kin=None, euclidean=False)`

A Feynman diagram as a graph, from strings.

- **edges**: the lines, "A-B" between vertices A and B, or "A-B:m" for a line of mass m . **The order of the lines is the numbering** $x_1, x_2, \dots$ (the lecture's $\alpha_1, \alpha_2, \dots$).
- **legs**: the momentum **entering** at each vertex, for example {"A": "p", "B": "-p"}. They must add up to zero.
- **kin**: the scalar products of the external momenta. **euclidean=True** for Euclidean signs.
- returns a `FeynmanGraph` with these methods:
 - `.spanning_trees()`: the 1-trees, as tuples of the **erased** lines (counted from 0)
 - `.two_forests()`: the 2-trees, as pairs (erased lines, vertices of one of the two pieces)
 - `.U()`, `.F0()` ($= V$), `.F()`: the polynomials, read off the trees exactly by the rules above
 - `.plot()`: a picture
 - `.N`, `.V`, `.L`: the numbers of lines, vertices and loops

FEYNSAGE FUNCTION `draw_panels(g, erased, titles=None, ncols=5)`

Draws the graph g several times side by side, each copy with its own erased lines (grey and dashed). Feed it `g.spanning_trees()` to see all the 1-trees.

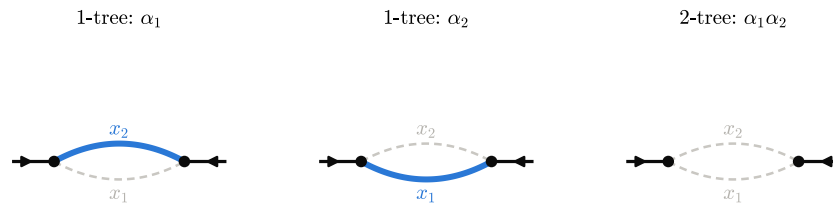
8.1 The bubble with two masses

Two 1-trees (erase the top line or the bottom line) and one 2-tree (erase both lines, then p flows between the two vertices).

```
In [32]: from feynsage.plotting import draw_panels
xs = lambda rem: "$" + "".join("\\alpha_{%d}" % (i + 1) for i in rem) + "$"
g2 = graph("A-B:m1, A-B:m2", {"A": "p", "B": "-p"}, kin={"p^2": "pp"}, euclidean=True)
trees = g2.spanning_trees(); forests = [r for r, c in g2.two_forests()]
titles = ["1-tree: " + xs(r) for r in trees] + ["2-tree: " + xs(r) for r in forests]
fig = draw_panels(g2, trees + forests, titles=titles, momenta=False, ncols=3, size=1.9)
print("U =", g2.U())
print("V =", g2.F0())
print("F = V + (m1^2 x1 + m2^2 x2) U =", g2.F())
```

```
U = x1 + x2
V = pp*x1*x2
F = V + (m1^2 x1 + m2^2 x2) U = m1^2*x1^2 + (m1^2 + m2^2 + pp)*x1*x2 + m2^2*x2^2
```

Out [32]:



So $\mathcal{U} = \alpha_1 + \alpha_2$, $V = \alpha_1 \alpha_2 p^2$ and $\mathcal{F} = V + (m_1^2 \alpha_1 + m_2^2 \alpha_2) \mathcal{U}$, as in the lecture.

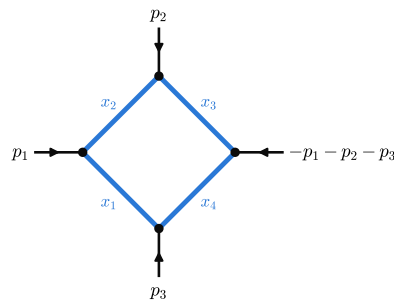
8.2 The box (Smirnov, Fig. 3.6)

The massless box with $p_i^2 = 0$, $s = (p_1 + p_2)^2$ and $t = (p_1 + p_3)^2$, with the lines numbered as in Smirnov's Fig. 3.6 (2006). Line 1 is on top (between p_1 and p_3), 2 on the left (between p_1 and p_2), 3 at the bottom (between p_2 and p_4) and 4 on the right (between p_3 and p_4). feynsage draws the square turned by 45 degrees.

```
In [33]: # Smirnov (2006) Fig. 3.6: line 1 on top, 2 on the left, 3 at the bottom, 4 on the right;
# p1 and p2 enter on the left, p3 and p4 on the right, s = (p1 + p2)^2, t = (p1 + p3)^2
box = graph("TL-TR, TL-BL, BL-BR, TR-BR",
            {"TL": "p1", "BL": "p2", "TR": "p3", "BR": "-p1-p2-p3"},
            kin={"p1^2": 0, "p2^2": 0, "p3^2": 0,
                "p1.p2": "s/2", "p1.p3": "t/2", "p2.p3": "-(s+t)/2"},
            euclidean=True)
fig = box.plot(figsize=(2.8, 2.8))
print("U =", box.U(), " V =", box.F0())
```

```
U = x1 + x2 + x3 + x4      V = s*x1*x3 + t*x2*x4
```

Out [33]:

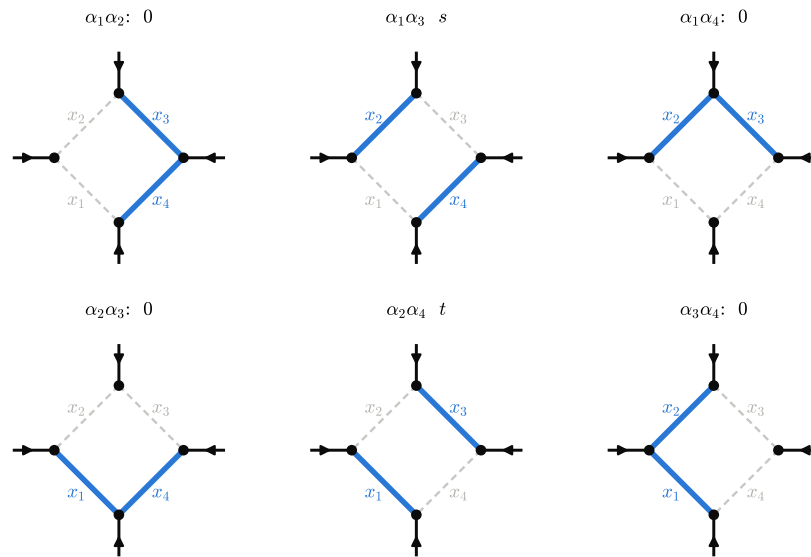


All six 2-trees. Only the two that cut the box into a left and a right half, or a top and a bottom half, carry momentum. The other four cut off a single corner, which receives only one p_i with $p_i^2 = 0$, so they do not contribute (the grey pictures of the lecture). The small helper `momentum_in` adds up the momenta entering one piece:

```
In [34]: def momentum_in(g, comp):
    tot = {}
    for v in comp:
        for k, c in g.ext.get(v, {}).items():
            tot[k] = tot.get(k, 0) + c
    return {k: c for k, c in tot.items() if c}

ft = box.two_forests()
titles = []
for rem, comp in ft:
    P2 = box.kin.square_external(momentum_in(box, comp))
    titles.append(xs(rem) + ("": 0" if P2 == 0 else " $s$" % latex(P2)))
fig = draw_panels(box, [r for r, c in ft], titles=titles, momenta=False, ncols=3,
                    size=2.0)
```

Out[34]:



THE LABELS MATTER

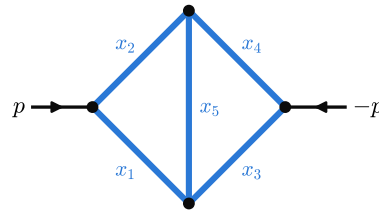
Erasing the top and bottom lines 1 and 3 separates p_1, p_2 from p_3, p_4 and gives $s \alpha_1 \alpha_3$. Erasing the left and right lines 2 and 4 gives $t \alpha_2 \alpha_4$. So $V = s \alpha_1 \alpha_3 + t \alpha_2 \alpha_4$, as on the board and in Smirnov's eq. (3.28). His eq. (3.27) prints s and t the other way round, a misprint that does no harm when all powers are 1. The safe rule is to look at the picture. s goes with the pair of lines whose removal separates p_1, p_2 from p_3, p_4 .

8.3 Smirnov's two-loop example

This is the two-loop propagator diagram of the lecture, Smirnov's Fig. 3.10 and Example 3.5 (2006), with the lines numbered as in the book. Lines 1 and 2 are on the left, 3 and 4 on the right and 5 in the middle. The polynomials below are his eqs. (3.44) and (3.45).

```
In [35]: kite = graph("L-B, L-T, B-R, T-R, T-B", {"L": "p", "R": "-p"}, kin={"p^2": "pp"},
          euclidean=True)
fig = kite.plot(figsize=(2.8, 2.8))
```

Out[35]:

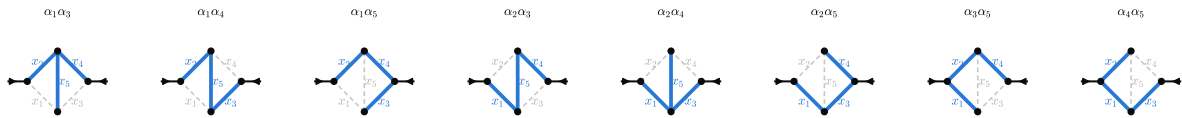


Eight 1-trees, each erasing two lines, give $\mathcal{U}$ of degree 2:

```
In [36]: trees = kite.spanning_trees()
fig = draw_panels(kite, trees, titles=[xs(r) for r in trees], momenta=False, ncols=8,
                  size=1.5)
print(len(trees), "1-trees: U =", kite.U())
```

8 1-trees: $U = x1*x3 + x2*x3 + x1*x4 + x2*x4 + x1*x5 + x2*x5 + x3*x5 + x4*x5$

Out[36]:



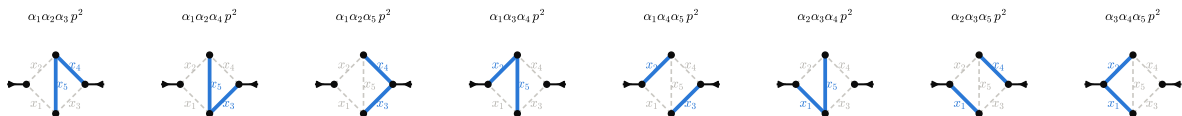
Eight 2-trees carry the momentum p , each erasing three lines:

```
In [37]: carry = [r for r, c in kite.two_forests() if momentum_in(kite, c)]
fig = draw_panels(kite, carry, titles=[xs(r) + "$\\,p^2$" for r in carry], momenta=False,
                  ncols=8, size=1.5)
print(len(carry), "2-trees with p^2:")
terms = str(kite.F()).split(" + ")
print("F =", " + ".join(terms[:4]))
print(" +", " + ".join(terms[4:]))
```

8 2-trees with p^2 :

$F = pp*x1*x2*x3 + pp*x1*x2*x4 + pp*x1*x3*x4 + pp*x2*x3*x4$
 $+ pp*x1*x2*x5 + pp*x2*x3*x5 + pp*x1*x4*x5 + pp*x3*x4*x5$

Out[37]:



These are the 1-trees and 2-trees drawn in the lecture.

8.4 From $\mathcal{U}$ and $\mathcal{F}$ back to the integral

Put $\mathcal{U}$ and $\mathcal{F}$ into the general formula. For the massless bubble ($N = 2$, $L = 1$) the delta function leaves one integral over x . `SR(str(...))` turns the polynomials into ordinary expressions so that $x_1 = x$, $x_2 = 1 - x$ can be put in. At $D = 3.3$ and $p^2 = 1$ the result agrees with the closed form `bubble_massless(1, 1, 1)` (to the precision of the numerical integration):

```
In [38]: # the massless bubble from U and F with the general formula:
# N = 2 lines, L = 1 loop, powers 1
from feynsage.oneloop import bubble_massless
g0 = graph("A-B, A-B", {"A": "p", "B": "-p"}, kin={"p^2": 1}, euclidean=True)
U0, F0 = g0.U(), g0.F()
Dv, N, L = 3.3, 2, 1
xx = var('xx')
u = SR(str(U0)).subs(x1=xx, x2=1 - xx); f = SR(str(F0)).subs(x1=xx, x2=1 - xx)
val = gamma(N - L*Dv/2) * numerical_integral(u^(N - (L + 1)*Dv/2)/f^(N - L*Dv/2), 0, 1)[0]
print("U =", U0, " F =", F0)
print("from U and F:", val, " closed form:", bubble_massless(1, 1, 1).subs(D=Dv).n())

U = x1 + x2      F = x1*x2
from U and F: 5.44044255456555      closed form: 5.44044257084738
```

8.5 Sir's notebook: completing the square

The Mathematica notebook shown in the lecture finds $\mathcal{U}$ and $\mathcal{F}$ in a second way, without drawing any trees. The formula it uses was not written on the board, so here it is step by step.

Step 1: one denominator. With Feynman parameters x_i all the propagators are combined into one sum, $\sum_i x_i D_i$. Each D_i is quadratic in the loop momentum, so the sum is too.

Step 2: sort it by powers of the loop momentum. For one loop with loop momentum ℓ every quadratic expression can be written as

$$\sum_i x_i D_i = M \ell^2 - 2Q \cdot \ell + J$$

- M collects everything in front of ℓ^2 . Every propagator contains ℓ^2 once, so $M = \sum_i x_i$.
- Q is a momentum. It collects the terms linear in ℓ (the factor -2 is a convention that makes the next step neat).
- J is the rest, made of external momenta and masses with no ℓ at all.

Step 3: complete the square. Shift $\ell = \ell' + Q/M$:

$$M \ell^2 - 2Q \cdot \ell + J = M \ell'^2 + (J - (Q \cdot Q)/M)$$

Now the integral over ℓ' is a tadpole, because the loop momentum appears only as ℓ'^2 , with a “mass” $(J - Q^2/M)/M$. Doing it gives a power of M and a power of $J - Q^2/M$ and they combine exactly into the general formula above with

$$\mathcal{U} = M = \det M, \quad \mathcal{F} = \det M (J - Q M^{-1} Q)$$

At more loops there are several loop momenta $\ell_1, \ell_2, \dots$. The coefficients of $\ell_r \cdot \ell_s$ form a matrix M (2 by 2 at two loops) and Q becomes a list of momenta (one per loop). The same formulas hold with $\det M$ and the matrix inverse M^{-1} . This is why the notebook writes them with a determinant.

As a worked example take the bubble with two masses (Euclidean, $D_1 = k^2 + m_1^2$, $D_2 = (k + p)^2 + m_2^2$). Write the scalar products as symbols ($kk = k^2$, $kp = k \cdot p$, $pp = p^2$), expand and read off M , Q and J as the coefficients.

```
In [39]: # the two-mass bubble, Euclidean: D1 = k^2 + m1^2, D2 = (k + p)^2 + m2^2
# write the scalar products as symbols: kk = k.k, kp = k.p, pp = p.p
x1, x2, m1, m2, kk, kp, pp = var('x1 x2 m1 m2 kk kp pp')
S = (x1*(kk + m1^2) + x2*(kk + 2*kp + pp + m2^2)).expand()
M = S.coefficient(kk)
Qc = -S.coefficient(kp)/2
J = S.subs(kk=0, kp=0)
print("sum x_i D_i =", S)
print("M =", M, " Q = (%s) p" % Qc, " J =", J)
U_sq = M
# det M (J - Q.M^-1.Q), with Q.Q = Qc^2 p.p
F_sq = (M*(J - Qc^2*pp/M)).simplify_rational().expand()
print("U = det M =", U_sq)
print("F = det M (J - Q M^-1 Q) =", F_sq.collect(pp))

# sum_i x_i D_i
# the coefficient of k.k
# -2 Q.k: Q = Qc * p
# what has no loop momentum
```

```

sum x_i D_i = m1^2*x1 + m2^2*x2 + kk*x1 + kk*x2 + 2*kp*x2 + pp*x2
M = x1 + x2      Q = (-x2) p      J = m1^2*x1 + m2^2*x2 + pp*x2
U = det M = x1 + x2
F = det M (J - Q M^-1 Q) = m1^2*x1^2 + m1^2*x1*x2 + m2^2*x1*x2 + m2^2*x2^2 + pp*x1*x2

```

This is the same $\mathcal{F}$ as from the 1-trees and 2-trees above.

FEYNSAGE FUNCTION `fam.UF()`

$\mathcal{U}$ and $\mathcal{F}$ of a family by exactly this method. It builds M , Q and J from the list of propagators and returns $\det M$ and $\det M(J - QM^{-1}Q)$, for any number of loops (no graph needed). For a Minkowski family the signs are those of the notebook.

```

In [40]: fam2 = family(["k", "m1"), ("k + p", "m2"), kin={"p^2": "pp"}, euclidean=True)
U_f, F_f = fam2.UF()
print("fam.UF() gives the same:      ", bool(SR(str(U_f)) == U_sq),
      bool((SR(str(F_f)) - F_sq).expand() == 0))
print("and the 2-trees gave the same:", bool((SR(str(g2.F())) - F_sq).expand() == 0))

```

```

fam.UF() gives the same:      True True
and the 2-trees gave the same: True

```

The notebook's own examples, the triangle, the bubble with two masses and the box (Minkowski signs, as in the notebook), come out as on the screen in the lecture:

```

In [41]: # sir's notebook completes the square, sum x_i D_i = l.M.l - 2 Q.l + J:
# U = det M, F = det M (J - Q.M^-1.Q)
tri = family(["k", "k + p1", "k + p1 + p2"],
             kin={"p1^2": "P1Sq", "p2^2": "P2Sq", "p1.p2": "(Q5q - P1Sq - P2Sq)/2"})
bb = family(["k", "m1"), ("k + p1", "m2"), kin={"p1^2": "p1sq"})
bx = family(["k", "k + p1", "k + p1 + p2", "k - p3"],
             kin={"p1^2": 0, "p2^2": 0, "p3^2": 0,
                  "p1.p2": "s/2", "p1.p3": "t/2", "p2.p3": "-(s+t)/2"})
for name, fam in [("triangle", tri), ("bubble", bb), ("box", bx)]:
    U_, F_ = fam.UF()
    print("%-9s U = %-18s F = %s" % (name, U_, F_))

triangle U = x1 + x2 + x3      F = (-P1Sq)*x1*x2 + (-Q5q)*x1*x3 + (-P2Sq)*x2*x3
bubble   U = x1 + x2          F = m1^2*x1^2 + (m1^2 + m2^2 - p1sq)*x1*x2 + m2^2*x2^2
box      U = x1 + x2 + x3 + x4 F = (-s)*x1*x3 + (-t)*x2*x4

```

9 A real prediction: the decay rate of the neutral pion

Now put the tools together for a number that can be measured in a laboratory, the rate at which the neutral pion decays into two photons. It has no charge, so the photons cannot attach to it directly. The decay goes through a **triangle** of charged quarks. The physics is explained below as we go and every step is done with feynsage, with the graph method of Part 2 at its centre.

THE MODEL

The quarks u and d ($Q_u = 2/3$, $Q_d = -1/3$) come in N_c colours. They couple to the photon with charge Qe and to the pion through $g\bar{q}\gamma_5\tau_3 q\pi^0$ ($\tau_3 = +1$ for u , -1 for d), with $g = m/f_\pi$ (the Goldberger-Treiman relation, $f_\pi \approx 92$ MeV). For each quark there are two triangle diagrams, one for each order of the photons. The photons have $k_1^2 = k_2^2 = 0$ and the pion $(k_1 + k_2)^2 = m_\pi^2$.

9.1 Step 1: the graph, its trees and its polynomials

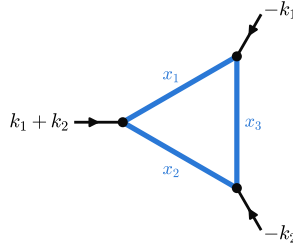
Write the triangle as a graph. The pion vertex is P and the photon vertices are A and B. All three lines are quarks of mass m . Line 1 is P-A (parameter x_1), line 2 is B-P (x_2) and line 3 is A-B (x_3). The pion brings in $k_1 + k_2$ at P and the photons take out k_1 at A and k_2 at B (Minkowski space, $k_1^2 = k_2^2 = 0$, $2k_1 \cdot k_2 = m_\pi^2$):

```
In [42]: # the quark triangle: P = pion vertex, A and B = photon vertices;
# all three lines are quarks of mass mq
# line 1 = P-A (Feynman parameter x), 2 = B-P (y), 3 = A-B (1 - x - y); Minkowski signs
tri = graph("P-A:mq, B-P:mq, A-B:mq", {"P": "k1 + k2", "A": "-k1", "B": "-k2"},
          kin={"k1^2": 0, "k2^2": 0, "k1.k2": "mpi2/2"})
fig = tri.plot(figsize=(2.6, 2.6))
print("U =", tri.U())
print("F =", tri.F())
```

$$U = x_1 + x_2 + x_3$$

$$F = mq^2 x_1^2 + (2mq^2 - mpi^2)x_1 x_2 + mq^2 x_2^2 + 2mq^2 x_1 x_3 + 2mq^2 x_2 x_3 + mq^2 x_3^2$$

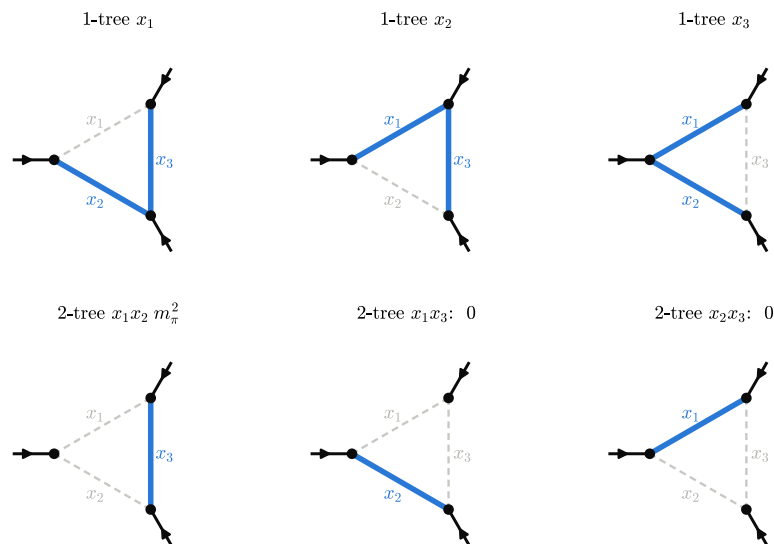
Out [42]:



Now the rules of Part 2, drawn. One loop, so a 1-tree erases one line and a 2-tree erases two. Each 2-tree cuts off one vertex. Only the one that cuts off the pion vertex P carries momentum, $(k_1 + k_2)^2 = m_\pi^2$. The two that cut off a photon vertex carry $k_1^2 = 0$ or $k_2^2 = 0$ and drop out:

```
In [43]: trees = tri.spanning_trees(); two = tri.two_forests()
lab = lambda rem: "$" + "".join("x_{%d}" % (i + 1) for i in rem) + "$"
def term(rem, comp):
    P2 = tri.kin.square_external(momentum_in(tri, comp))
    return lab(rem) + (";\m_\pi^2$" if P2 != 0 else ": 0")
fig = draw_panels(tri, trees + [r for r, c in two],
                  titles=["1-tree " + lab(r) for r in trees]
                    + ["2-tree " + term(r, c) for r, c in two],
                  momenta=False, ncols=3, size=2.0)
```

Out [43]:



So $\mathcal{U} = x_1 + x_2 + x_3$ and $V = x_1 x_2 m_\pi^2$. With the mass term $\mathcal{F} = -V + \mathcal{U}(x_1 + x_2 + x_3)m^2$ (Minkowski sign), which is the $\mathcal{F}$ printed above. On the simplex $x_1 + x_2 + x_3 = 1$, with $x_1 = x$ and $x_2 = y$, the polynomials become $\mathcal{U} = 1$ and

```
In [44]: x, y = var('x y')
m, m_pi = var('m m_pi')
Delta = SR(str(tri.F())).subs(x1=x, x2=y, x3=1 - x - y).expand() # U = 1 on the simplex
Delta = Delta.subs({SR.var('mq'): m, SR.var('mpi2'): m_pi^2})
Delta
```

```
Out[44]: -m_pi^2*x*y + m^2
```

This is the Δ of the textbook calculation, read off the trees without completing any square.

9.2 Step 2: the integral from $\mathcal{U}$ and $\mathcal{F}$

Now put $\mathcal{U}$ and $\mathcal{F}$ into the general formula of Part 2. Here $N = 3$ lines, all with power 1, $L = 1$ loop and $D = 4$ (the triangle is finite):

$$\int \frac{d^4\ell}{i\pi^2} \frac{1}{D_1 D_2 D_3} = (-1)^3 \Gamma(3-2) \int_0^\infty dx_1 dx_2 dx_3 \delta(1-x_1-x_2-x_3) \frac{\mathcal{U}^{3-4}}{\mathcal{F}^{3-2}} = - \int d^3x \delta(\dots) \frac{1}{\mathcal{U}\mathcal{F}}$$

feynsage writes the integrand straight from the polynomials of Step 1:

```
In [45]: # the general formula with N = 3 lines (powers 1), L = 1 loop, D = 4, Minkowski:
# Int d^4l/(i pi^2) 1/(D1 D2 D3)
# = (-1)^N Gamma(N - L D/2) Int dx delta(1 - sum x) U^(N-(L+1)D/2) / F^(N-L D/2)
N, L, Dv = 3, 1, 4
print("prefactor (-1)^N Gamma(N - L D/2) =", (-1)^N*gamma(N - L*Dv/2),
      " power of U:", N - (L + 1)*Dv/2, " power of F:", -(N - L*Dv/2))
Ux, Fx = SR(str(tri.U())), SR(str(tri.F()))
integrand = (-1)^N*gamma(N - L*Dv/2) * Ux^(N - (L + 1)*Dv/2) / Fx^(N - L*Dv/2)
integrand = integrand.subs(x3=1 - x1 - x2)
integrand = integrand.subs({SR.var('mq'): m, SR.var('mpi2'): m_pi^2}).simplify_full()
print("integrand on the simplex x1 + x2 + x3 = 1:", integrand)

prefactor (-1)^N Gamma(N - L D/2) = -1 power of U: -1 power of F: -1
integrand on the simplex x1 + x2 + x3 = 1: 1/(m_pi^2*x1*x2 - m^2)
```

That is $-1/\Delta$ on the simplex, so the triangle is $-\int_0^1 dx \int_0^{1-x} dy 1/(m^2 - xym_\pi^2) = -I(r)/m^2$ with

$$I(r) = \int_0^1 dx \int_0^{1-x} dy \frac{1}{1 - xyr}, \quad r = \frac{m_\pi^2}{m^2}$$

Do the integral numerically from $\mathcal{U}$ and $\mathcal{F}$ (here $m = 1$, $r = 0.5$) and compare with the closed form $I(r) = (2/r) \arcsin^2(\sqrt{r}/2)$:

```
In [46]: from scipy.integrate import dblquad
mv, rv = 1.0, 0.5 # m = 1 and m_pi^2 = r m^2 with r = 0.5
f_num = fast_callable(integrand.subs(m=mv, m_pi=sqrt(rv)),
                      vars=[SR.var('x1'), SR.var('x2')])
val = dblquad(lambda b, a: f_num(a, b), 0, 1, 0, lambda a: 1 - a)[0]
print("Int d^4l/(i pi^2) 1/(D1 D2 D3) from U and F: ", val)
print("-I(r)/m^2 with I(r) = (2/r) arcsin^2(sqrt(r)/2):",
      -(2/rv*arcsin(sqrt(rv)/2)^2)/mv^2)

Int d^4l/(i pi^2) 1/(D1 D2 D3) from U and F: -0.5223447929624236
-I(r)/m^2 with I(r) = (2/r) arcsin^2(sqrt(r)/2): -0.522344792962424
```

The amplitude uses the measure $d^4\ell/(2\pi)^4$, which is $i\pi^2/(2\pi)^4$ times the measure of the formula:

```
In [47]: Ir = var('I_r')
# the loop measure of the amplitude is d^4l/(2 pi)^4 = (i pi^2/(2 pi)^4) d^4l/(i pi^2)
(I*pi^2/(2*pi)^4 * (-Ir/m^2)).simplify_full()
```

```
Out[47]: -\frac{i I_r}{16 \pi^2 m^2}
```

For $I(r)$ itself, expand $1/(1 - xyr)$ in powers of r , integrate term by term and compare with the closed form.

```
In [48]: r = var('r')
term_k = lambda k: integrate(integrate((x*y)^k, y, 0, 1-x), x, 0, 1)
series_I = sum(r^k * term_k(k) for k in range(10))
closed = 2/r * arcsin(sqrt(r)/2)^2
print("I(r) term by term: ", series_I.series(r, 4).truncate())
print("closed form, series:", closed.taylor(r, 0, 3))
print("heavy quark, r -> 0:", limit(closed, r=0))
print("m = 330 MeV: 2 I(r) =", (2*closed.subs(r=(134.9768/330)^2)).n(digits=5))
```

```
I(r) term by term: 1/1120*r^3 + 1/180*r^2 + 1/24*r + 1/2
closed form, series: 1/1120*r^3 + 1/180*r^2 + 1/24*r + 1/2
heavy quark, r -> 0: 1/2
m = 330 MeV: 2 I(r) = 1.0143
```

For a heavy quark $I \rightarrow 1/2$, the area of the triangle. Even for a light constituent quark ($m = 330$ MeV) $2I(r)$ is only 1.4% above 1.

9.3 Step 3: the Dirac trace

The numerator of the diagram is a trace of Dirac matrices. The pion vertex gives γ_5 , each photon vertex a γ^μ or γ^ν and each quark propagator its numerator $\gamma \cdot p + m$, where $\gamma \cdot p = \gamma^\mu p_\mu$ (often written with a slash through p). With the momenta of the triangle and the loop momentum shifted to q ($\ell = q - xk_1 + yk_2$, as in Step 2) the trace is

$$\text{Tr}[\gamma_5 (\gamma \cdot (q - xk_1 - (1-y)k_2) + m) \gamma^\nu (\gamma \cdot (q - xk_1 + yk_2) + m) \gamma^\mu (\gamma \cdot (q + (1-x)k_1 + yk_2) + m)]$$

FEYNSAGE FUNCTION `dirac_trace(expr, dim=4, rules=None)`

The trace of a product of Dirac matrices, done by the program FORM. The product is written as on paper, with $*$ between the matrices in their order:

<code>q, k1 = momenta("q k1")</code>	declares the momenta
<code>mu, nu = lorentz_indices("mu nu")</code>	declares the indices
<code>gamma(mu)</code>	γ^μ
<code>slash(q)</code>	$\gamma \cdot q = \gamma^\mu q_\mu$
<code>slash(q - x*k1) + m</code>	$\gamma \cdot (q - xk_1) + m$ (m times the unit matrix)
<code>gamma5()</code>	γ_5

- `dim=4`: four dimensions, needed with γ_5 . A symbol such as `d` gives d dimensions
- every repeated index is summed. The answer is a Sage expression with $p \cdot q$, p^μ , $g^{\mu\nu}$ and the usual Levi-Civita symbol $\varepsilon^{\mu\nu\rho\sigma}$ of Peskin and Schroeder ($\varepsilon^{0123} = +1$, so $\text{Tr}[\gamma^\mu \gamma^\nu \gamma^\rho \gamma^\sigma \gamma_5] = -4i\varepsilon^{\mu\nu\rho\sigma}$, their eq. (5.5)). In ε a momentum in a slot means it is contracted there, so $\varepsilon^{k_1 k_2 \mu \nu} = \varepsilon^{\rho \sigma \mu \nu} k_{1\rho} k_{2\sigma}$.
- FORM itself never has to be written. feynsage finds the momenta and indices, writes the FORM program, runs it and reads the answer back.

```
In [49]: # the trace above, written as on paper; FORM does the work
q, k1, k2 = momenta("q k1 k2")
mu, nu = lorentz_indices("mu nu")
m = var('m')
dirac_trace(gamma5() * (slash(q - x*k1 - (1-y)*k2) + m) * gamma(nu)
            * (slash(q - x*k1 + y*k2) + m) * gamma(mu)
            * (slash(q + (1-x)*k1 + y*k2) + m))
```

Out [49]: $4im\varepsilon^{k_1 k_2 \mu \nu}$

So the whole numerator is one term,

$$4im\varepsilon^{k_1 k_2 \mu \nu} = 4im\varepsilon^{\mu\nu\rho\sigma} k_{1\rho} k_{2\sigma}$$

(moving the two momentum slots past the two index slots is an even permutation). Nothing depends on q , x or y , so the loop integral of Step 2 is all that is needed. It is a pure scalar triangle. Only the terms with exactly one m survive, because a trace with γ_5 needs an even number, at least four, of other γ matrices.

9.4 Step 4: the amplitude and why the quark mass drops out

Collect the factors of one quark. They are N_c colours, $(eQ)^2$ from the photon vertices, g from the pion vertex, $4m$ from the trace (Step 3), $I/(16\pi^2 m^2)$ from the integral (Step 2) and a factor 2 for the two orders of the photons. Then put in $g = m/f_\pi$ and add u and d (with $\tau_3 = \pm 1$):

```
In [50]: Nc, e, Q, g, f, mq_ = var('N_c e Q g f_pi m_q')
# one quark, both orderings of the photons, heavy quark
A_one = Nc * e^2 * Q^2 * g / (4*pi^2*mq_) * 2*limit(closed, r=0)
print("one quark:", A_one)
print("with g = m_q/f_pi:", A_one.subs(g=mq_/f))
alpha = var('alpha')
A = A_one.subs(g=mq_/f, Q=2/3) - A_one.subs(g=mq_/f, Q=-1/3) # u minus d
A = A.subs(e=sqrt(4*pi*alpha))
print("u and d, e^2 = 4 pi alpha:", A.simplify_full())

one quark: 1/4*N_c*Q^2*e^2*g/(pi^2*m_q)
with g = m_q/f_pi: 1/4*N_c*Q^2*e^2/(pi^2*f_pi)
u and d, e^2 = 4 pi alpha: 1/3*N_c*alpha/(pi*f_pi)
```

The m of the coupling cancels the $1/m$ of the loop. A heavier quark couples more strongly and propagates less. The answer knows only the charges and the number of colours, $A = N_c(Q_u^2 - Q_d^2)e^2/(4\pi^2 f_\pi) = \alpha/(\pi f_\pi)$ for $N_c = 3$. This is the axial anomaly (Adler 1969).

9.5 Step 5: the decay rate

Square the amplitude and sum over the photon polarisations. That needs the contraction $\varepsilon^{\mu\nu\rho\sigma}\varepsilon_{\mu\nu\alpha\beta}k_{1\rho}k_{2\sigma}k_1^\alpha k_2^\beta$. It is written exactly as on paper, with the momenta put straight into the slots of ε . `contract` turns it into a FORM program, runs it and reads the answer back (debug=True prints the program it made):

FEYNSAGE FUNCTION `contract(expr, rules=None, dim=4, debug=False)`
Sums every repeated Lorentz index of an expression made of `dot(p, q)` ($p \cdot q$), `comp(p, mu)` (p^μ), `metric(mu, nu)` ($g^{\mu\nu}$) and `epsilon(...)` ($\varepsilon^{\mu\nu\rho\sigma}$), done by FORM.

- `rules`: kinematics to put in, for example `{dot(k1, k1): 0, dot(k1, k2): s/2}`
- `dim`: 4, or a symbol like `d`
- returns a Sage expression. The same kinematics can also be put in afterwards with `.subs(...)`.

```
In [51]: # the contraction above, written as on paper
contract(epsilon(mu, nu, k1, k2) * epsilon(mu, nu, k1, k2),
        rules={dot(k1, k1): 0, dot(k2, k2): 0}, debug=True)
```

```
--- FORM program ---
Vectors k1,k2;
Indices mu,nu;
Off statistics;
Local FSE1 = ((i_*e_(k1,k2,mu,nu)))^2;
contract;
id k1.k1 = 0;
id k2.k2 = 0;
.sort
id k1.k1 = 0;
id k2.k2 = 0;
Format nospaces;
Print;
.end

--- FORM output ---

FSE1=2*k1.k2^2;
```

```
Out [51]: 2 (k1 · k2)2
```

The printed program shows what was done for us. It has FORM's e_- for ε (times i , because FORM's symbol is $-i\varepsilon$), the vectors and indices declared, `contract` and the two rules $k_1^2 = k_2^2 = 0$. The answer is $\varepsilon^{\mu\nu\rho\sigma}\varepsilon_{\mu\nu\alpha\beta}k_{1\rho}k_{2\sigma}k_1^\alpha k_2^\beta = 2(k_1 \cdot k_2)^2 = m_\pi^4/2$.

WRITING FORM YOURSELF

The older interface `form.compute`, with Sage notation in strings, is still there. For anything else `form.run_form(code)` runs a FORM program written by hand and returns FORM's output. A pedestrian introduction to FORM for particle physics (declarations, `g_` and `trace4`, `e_` and `contract`, `id` and a full Compton-scattering calculation) is on my blog, rousan.netlify.app/pages/physics/blogs/form_pedestrian_qft_blog.

With the two-body phase space $1/(8\pi)$, a factor $1/2$ for the two identical photons and $1/(2m_\pi)$ for the decaying pion,

$$\Gamma = \frac{1}{2m_\pi} \cdot \frac{1}{2} \cdot \frac{1}{8\pi} \cdot A^2 \frac{m_\pi^4}{2} = \frac{A^2 m_\pi^3}{64\pi}$$

Now the numbers, with $\alpha = 1/137.036$, $m_{\pi^0} = 134.977$ MeV and $f_\pi = 130.2/\sqrt{2}$ MeV:

```
In [52]: mpi = var('m_pi')
Gamma = (A^2 * mpi^3 / (64*pi)).simplify_full()
print("Gamma =", Gamma)
vals = {alpha: 1/137.035999, mpi: 134.9768, f: 130.2/sqrt(2)} # MeV
G3 = (Gamma.subs(vals).subs({Nc: 3}) * 1e6).n() # eV
G1 = (Gamma.subs(vals).subs({Nc: 1}) * 1e6).n()
print("width feynsage %.2f eV" % G3)
print("with N_c = 1: %.2f eV, nine times too small" % G1)
vals_f = dict(vals); vals_f[f] = 1.01*130.2/sqrt(2)
Gf = (Gamma.subs(vals_f).subs({Nc: 3})*1e6).n()
print("f_pi 1%% larger: Gamma = %.2f eV, 2%% lower (Gamma ~ 1/f_pi^2)" % Gf)

Gamma = 1/576*N_c^2*alpha^2*m_pi^3/(pi^3*f_pi^2)
width feynsage 7.79 eV
with N_c = 1: 0.87 eV, nine times too small
f_pi 1% larger: Gamma = 7.63 eV, 2% lower (Gamma ~ 1/f_pi^2)
```

RESULT: THE PION DECAY RATE FROM ONE TRIANGLE

	feynsage
width $\Gamma(\pi^0 \rightarrow \gamma\gamma)$, $N_c = 3$	7.79 eV
the same with $N_c = 1$	0.87 eV

READING THE RESULT

- The prediction has its own small uncertainty. Since $\Gamma \propto 1/f_\pi^2$, 1% in f_π moves it by 2%.
- With one colour the rate would be nine times smaller. Adler already saw this mismatch in 1969. From the measured lifetime he found $|S| = 0.44$ for the number $S = \sum_j g_j Q_j^2$ that multiplies the amplitude, while the quark model with one colour gives $S = 1/6$, about three times too small. Three colours give $S = 1/2$.
- The quark mass does not matter (Step 4), which is the content of the anomaly.

10 From a model to a cross section: diagrams like FeynArts

FeynArts makes Feynman diagrams from a model and FeynCalc squares them. feynsage now does both. We follow a FeynArts and FeynCalc notebook step by step: $e^-e^+ \rightarrow \mu^-\mu^+$ in the Standard Model at tree level, with the lepton masses kept.

10.1 Topologies

`topologies(loops, n_in, n_out)` makes every way of joining the external legs with vertices of three or four lines, like `CreateTopologies`. For $2 \rightarrow 2$ at tree level there are four: one quartic vertex and the s , t and u channels. The counts for $2 \rightarrow 3$ at tree level, $2 \rightarrow 2$ at one loop and $1 \rightarrow 1$ at two loops are those of FeynArts (25, 99 and 29):

```
In [53]: tops = topologies(0, 2, 2) # tree level, 2 -> 2
len(tops), [len(topologies(L, a, b)) for (L, a, b) in ((0, 2, 3), (1, 2, 2), (2, 1, 1))]
```

```
Out[53]: (4, [25, 99, 29])
```

10.2 Putting fields on the lines

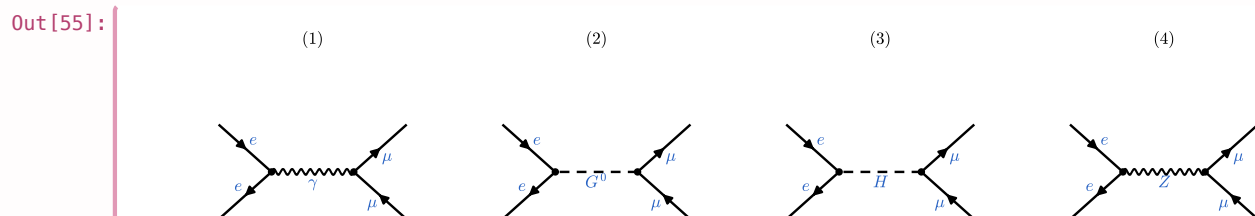
`insert_fields` puts the fields of a model on every line so that each vertex is a vertex of the model, like `InsertFields`. `SM()` is the Standard Model with QCD of FeynArts (the files `SM.mod` and `SMQCD.mod`, in the conventions of Denner), in Feynman gauge. Fields are written as "e-", "e+", "mu-", "A", "Z", "W+", "H", "G0", "g", "u", "u~" and so on. Only the s channel gets fields: the photon, the Z , the Higgs boson and the neutral Goldstone boson G^0 :

```
In [54]: sm = SM() # FeynArts' {"SM", "SMQCD"}, Feynman gauge
diags = insert_fields(tops, ["e-", "e+"], ["mu-", "mu+"], sm)
diags.counts(), [d.propagator_fields() for d in diags]
```

```
Out[54]: ([0, 4, 0, 0], [[A], [G0], [H], [Z]])
```

The diagrams, drawn like FeynArts' Paint:

```
In [55]: fig = diags.draw(size=1.9)
```



At one loop, without tadpoles and self-energies on the external legs, there are 388 diagrams, again as in FeynArts:

```
In [56]: one_loop = insert_fields(topologies(1, 2, 2, exclude=("tadpoles", "wf")), ["e-", "e+"],
["mu-", "mu+"], sm)
len(one_loop)
```

```
Out[56]: 388
```

10.3 The squared amplitude

`diags.squared(p_in, p_out)` builds the amplitude of every diagram in the notation of this tutorial (spinors, γ^μ , P_L , P_R , propagators), adds them, multiplies by the complex conjugate and sums over the spins with FORM. We put in the Mandelstam variables and divide by 4 for the average over the initial spins:

```
In [57]: from feynsage.models import EL, SW, CW, MW, MZ, MH, ME, MM
p1, p2, p3, p4 = momenta("p1 p2 p3 p4")
s, t, u = var('s t u')
mandelstam = {dot(p1, p1): ME^2, dot(p2, p2): ME^2, dot(p3, p3): MM^2, dot(p4, p4): MM^2,
              dot(p1, p2): (s - 2*ME^2)/2, dot(p3, p4): (s - 2*MM^2)/2,
              dot(p1, p3): (ME^2 + MM^2 - t)/2, dot(p2, p4): (ME^2 + MM^2 - t)/2,
              dot(p1, p4): (ME^2 + MM^2 - u)/2, dot(p2, p3): (ME^2 + MM^2 - u)/2}
M2 = (diags.squared([p1, p2], [p3, p4]) / 4).subs(mandelstam) # spins summed, 1/4 for
the average
len(M2.expand().operands())
```

```
Out[57]: 104
```

That is 104 terms. At $\sqrt{s} = 70$ GeV and $\cos\theta = \pm 1/2$ (the same numbers as in the FeynCalc notebook):

```
In [58]: values = {EL: sqrt(4*pi/137.036), ME: 0.000511, MM: 0.105658, MZ: 91.1876, MW: 80.379, MH:
125.25,
              SW: sqrt(0.23122), CW: sqrt(1 - 0.23122)}
def point(rs, c, me=0.000511, mmu=0.105658):
    sv = rs^2; pin = sqrt(sv/4 - me^2); pout = sqrt(sv/4 - mmu^2)
    tv = me^2 + mmu^2 - sv/2 + 2*pin*pout*c
    return {s: sv, t: tv, u: 2*me^2 + 2*mmu^2 - sv - tv}
[M2.subs(values).subs(point(70, c)).n(digits=9) for c in (0.5, -0.5)]
```

```
Out[58]: [0.00472095318, 0.0215928503]
```

Both agree with FeynCalc. They differ from each other because the Z makes the muons go more one way than the other (a forward-backward asymmetry).

A CHECK THAT NEEDS NO OTHER PROGRAM

In unitary gauge the Goldstone boson G^0 is absent and the Z propagator has an extra $k^\mu k^\nu / M_Z^2$. The physics must be the same. With $M_W = M_Z \cos\theta_W$ the difference is exactly zero:

```
In [59]: M2u = (diags.squared([p1, p2], [p3, p4], gauge="unitary") / 4).subs(mandelstam)
(M2u - M2).subs({MW: MZ*CW}).subs({SW: sqrt(1 - CW^2)}).subs(u=2*ME^2 + 2*MM^2 - s -
t).simplify_rational()
```

```
Out[59]: 0
```

10.4 The cross section, with the width of the Z

Near $\sqrt{s} = M_Z$ the Z propagator needs its width. `widths={"Z": GZ, "G0": GZ}` puts $q^2 - M^2 + iM\Gamma$ in the propagators before squaring. With $\Gamma_Z = 2.4952$ GeV, the cross section in pb at a few energies ($\sigma = (1/(32\pi s))(|p_\mu|/|p_e|) \int d\cos\theta |\mathcal{M}|^2$, and $1\text{GeV}^{-2} = 3.894 \times 10^8$ pb):

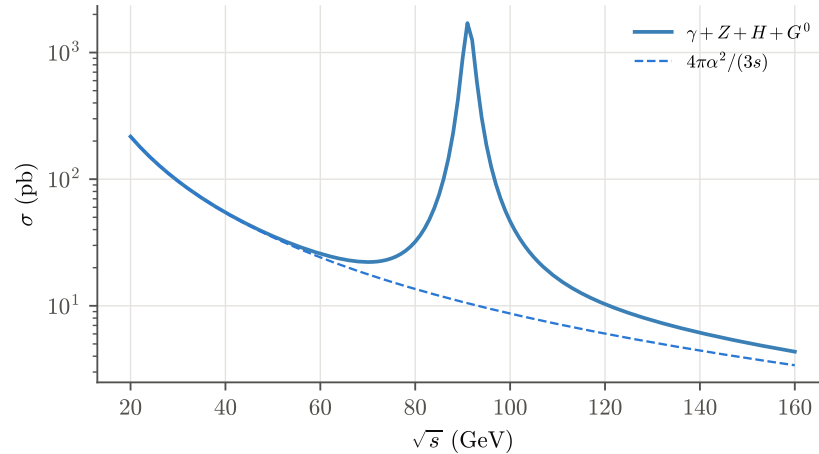
```
In [60]: GZ = var('GZ')
M2w = (diags.squared([p1, p2], [p3, p4], widths={"Z": GZ, "G0": GZ}) / 4).subs(mandelstam)
f = M2w.subs(values).subs({GZ: 2.4952})
c = var('c')
def sigma_pb(rs, me=0.000511, mmu=0.105658):
    sv = rs^2; pin = sqrt(sv/4 - me^2); pout = sqrt(sv/4 - mmu^2)
    g = fast_callable(f.subs(point(rs, c)), vars=[c], domain=CDF)
    return 3.894e8/(32*pi*n()*sv) * (pout/pin) * numerical_integral(lambda x: g(x).real(),
-1, 1)[0]
[round(sigma_pb(rs), 2) for rs in (20, 60, 91.1876, 160)]
```

```
Out[60]: [217.17, 25.81, 1754.64, 4.34]
```

Far below the Z the result is the QED value $4\pi\alpha^2/(3s)$ (217.15 pb at 20 GeV). At the peak it is about 1.75 nb:

```
In [61]: import numpy as np, matplotlib.pyplot as plt
roots = np.linspace(20, 160, 141)
fig, ax = plt.subplots(figsize=(5.2, 3.0))
ax.semilogy(roots, [sigma_pb(x) for x in roots], color="C3", lw=1.6, label=r"$\gamma + Z + H + G^0$")
ax.semilogy(roots, [4*np.pi/(3*x**2)/137.036**2*3.894e8 for x in roots], color="C0",
ls="--", lw=1, label=r"$4\pi\alpha^2/(3s)$")
ax.set_xlabel(r"$\sqrt{s}$ (GeV)"); ax.set_ylabel(r"$\sigma$ (pb)"); ax.legend(fontsize=8)
fig.tight_layout()
```

Out[61]:



WHERE THE WIDTH GOES

The FeynCalc notebook puts the width in after a partial fraction of $|\mathcal{M}|^2$ in s . Then the part of $|\mathcal{M}_Z|^2$ that is a polynomial in s is never divided by $(s - M_Z^2)^2 + M_Z^2\Gamma_Z^2$. Its cross section is about 0.1% too high far from the peak (217.40 pb at 20 GeV, where the photon alone gives 217.15 pb) and 0.7% too high at the peak. Putting the width in the propagator, as here, does not have this problem.

11 Processes in one line

The last chapter did everything by hand: the diagrams, the kinematics, the polarization and colour sums and the phase space. `process()` does all of it from one line written as in a textbook. The momenta are p_1, p_2 (in) and $p_3, p_4, \dots$ (out), $s = (p_1 + p_2)^2$, $t = (p_1 - p_3)^2$, $u = (p_1 - p_4)^2$, and θ is the angle between p_1 and p_3 in the centre-of-mass frame. `squared()` sums over the spins, polarizations and colours of all particles and divides by the number of initial states. Photons are summed with $-g_{\mu\nu}$, gluons with the physical sum and massive vectors with $-g_{\mu\nu} + k_\mu k_\nu / M^2$. QED $e^-e^+ \rightarrow \mu^-\mu^+$ with massless leptons gives $2e^4(t^2 + u^2)/s^2$, Peskin's (5.10) with massless muons:

```
In [90]: from feynsage.models import QED
P = process("e- e+ -> mu- mu+", model=QED(), massless=["e", "mu"])
P, P.squared().factor()
```

Out[90]:

$$\left(\text{process } e^- e^+ \rightarrow \mu^- \mu^+: 1 \text{ diagram}, \frac{2(t^2 + u^2)e^4}{s^2} \right)$$

One helicity state: `helicities=` with +1 or -1 for each fermion. Only the $e_R^- e_L^+$ and $e_L^- e_R^+$ states give anything, with $e^4(1 \pm \cos \theta)^2$ as in Peskin's (5.21) to (5.24) (p. 143):

```
In [91]: [(he, hm, P.squared(helicities={"e-": he, "e+": -he, "mu-": hm, "mu+": -hm}).factor()) for
he in (1, -1) for hm in (1, -1)]
```

Out[91]:

$$[(1, 1, e^4(\cos \theta + 1)^2), (1, -1, e^4(\cos \theta - 1)^2), (-1, 1, e^4(\cos \theta - 1)^2), (-1, -1, e^4(\cos \theta + 1)^2)]$$

The cross section: `dsigma_dcos()` puts in the phase space and `sigma()` integrates over the angle. Standard-Model numbers are built in $(\alpha(0))$, the masses of the PDG and $\sin^2 \theta_W = 1 - M_W^2/M_Z^2$, and keywords such as `alpha=1/128` change them. At $\sqrt{s} = 10$ GeV it is $4\pi\alpha^2/(3s)$ in pb:

```
In [92]: P.dsigma_dcos().factor(), P.sigma(sqrt_s=10, unit="pb"), float(4*pi*(1/137.035999084)^2/(3*10^2)*0.3893793721e9)
```

```
Out[92]: 
$$\left( \frac{(\cos^2 \theta + 1)e^4}{32\pi\sqrt{s}^2}, 868.5447687569202, 868.5447687569203 \right)$$

```

The full Standard Model (γ , Z , H and G^0 in the s channel, lepton masses kept), with the Z width in its propagator, across the Z peak (pb):

```
In [93]: Pz = process("e- e+ -> mu- mu+", widths={"Z": 2.4952}) # the full Standard Model, Z
with its width
[(rs, round(Pz.sigma(sqrt_s=rs, unit="pb"), 3)) for rs in (20, 60, 91.1876, 120, 200)]
```

```
Out[93]: [(20, 217.118), (60, 25.84), (91.18760000000000, 1867.875), (120, 10.643), (200, 2.655)]
```

Decays: `width()` for $1 \rightarrow 2$, symbolically and with the Standard-Model numbers (GeV):

```
In [94]: Hbb = process("H -> b b~")
G = Hbb.width()
print("Gamma(H -> b b~) divided by alpha m_h m_b^2 N_c (1 - 4 m_b^2/m_h^2)^(3/2)/(8 sin^2 m_W^2):",
      (G/(3*EL^2/(4*pi)*MH*MB^2*(1 - 4*MB^2/MH^2)^(3/2)/(8*SW^2*MW^2))).subs({MH: 125, MB:
5, MW: 80, SW: sqrt(0.23)}).n(digits=15))
for text in ("H -> b b~", "Z -> mu- mu+", "Z -> nu_e nu_e~", "W+ -> e+ nu_e", "t -> W+ b"):
    D = process(text)
    print("%-16s %.6f GeV" % (text, float(D.numeric(D.width()).n())))
```

```
Gamma(H -> b b~) divided by alpha m_h m_b^2 N_c (1 - 4 m_b^2/m_h^2)^(3/2)/(8 sin^2 m_W^2):
1.000000000000000
H -> b b~      0.004129 GeV
Z -> mu- mu+   0.080936 GeV
Z -> nu_e nu_e~ 0.160009 GeV
W+ -> e+ nu_e  0.219178 GeV
t -> W+ b      1.432814 GeV
```

Vector bosons and helicities: the longitudinal W^+W^- state of the next chapter but one, in one call:

```
In [95]: W = process("e- e+ -> W+ W-", massless=["e"])
LL = W.squared(helicities={"e-": -1, "e+": +1, "W+": 0, "W-": 0})
W.numeric(LL, sqrt_s=500, cos_theta=-1/2, alpha=1/(4*pi)).n(digits=20) # e = 1
```

```
Out[95]: 1.2168202717424543308
```

Quarks and gluons: $u\bar{u} \rightarrow gg$ averaged over the colours (1/9) and the spins (1/4), minus Xianyu's (17.23) (his solution of Peskin's Problem 17.3):

```
In [96]: Q = process("u u~ -> g g", exclude_fields=["A", "Z", "H", "G0", "G+", "W+"],
massless=["u"])
s, t, u = Q.s, Q.t, Q.u
from feynsage.models import GS
(Q.squared() - 32*GS^4/27*(t/u + u/t - 9*(t^2 + u^2)/(4*s^2))).subs(s=-t-u).simplify_rational()
```

```
Out[96]: 0
```

The products $\mathcal{M}_i \mathcal{M}_j^*$ of pairs of diagrams are worked out in parallel, one worker for each pair (`nproc=` sets the number, `nproc=1` works serially). With 16 workers $gg \rightarrow gg$ takes 12 s instead of 100 s and $e^+e^- \rightarrow W^+W^-$ 1.4 s instead of 3.4 s. `helicity_table()` makes the traces for different fermion helicities in parallel, and `loop_amplitude` reduces the one-loop diagrams in parallel.

12 Decays of the Higgs boson: Peskin's Final Project III

This is the final project of Part III of Peskin and Schroeder (pp. 775 to 777). The Higgs mass m_h is a free parameter and the other inputs are those of the book: $m_b = 5$ GeV, $m_t = 175$ GeV, $m_W = 80$ GeV, $m_Z = 91$ GeV, $\sin^2 \theta_w = 0.23$, $\alpha_s = 0.12$. The book does not fix α ; we take $\alpha = 1/128$. Every diagram below is made by `insert_fields` from the Standard Model of FeynArts, and every result is compared with the formula printed in the book or with Xianyu's worked solution.

12.1 (a) and (b): the decays at tree level

`width(final, masses)` is written once and used for every channel: the diagrams, $|\mathcal{M}|^2$ summed over spins, polarizations and colours, and the two-body phase space $\Gamma = p/(8\pi m_h^2) |\mathcal{M}|^2$:

```
In [62]: from feynsage.models import EL, SW, CW, MW, MZ, MH, MT, MB, GS
from feynsage.pv import finite_part, uv_part
from feynsage import metric as g_, comp as c_, dot as dot_
sm = SM()
k1, k2 = momenta("k1 k2") # the two decay products; the Higgs has k1 + k2
peskin = {EL: sqrt(4*pi/128), SW: sqrt(0.23), CW: sqrt(0.77), MW: 80, MZ: 91, MT: 175, MB: 5, GS: sqrt(4*pi*0.12)}
def width(final, masses, identical=False):
    # Gamma(h -> final) at tree level: diagrams, |M|^2 summed over spins, polarizations and colours
    dg = insert_fields(topologies(0, 1, 2), ["H"], final, sm)
    r = dg.squared([k1 + k2], [k1, k2])
    for leg in (2, 3):
        e = momenta("eps%d" % leg)
        if dg[0].model.field(final[leg - 2]).kind == "V":
            r = polarization_sum(r, e)
    r = color_factor(r, N=3)
    m1, m2 = masses
    r = r.subs({dot_(k1, k1): m1^2, dot_(k2, k2): m2^2, dot_(k1, k2): (MH^2 - m1^2 - m2^2)/2})
    p = sqrt((MH^2 - (m1 + m2)^2)*(MH^2 - (m1 - m2)^2))/(2*MH)
    return (p/(8*pi*MH^2) * r * (1/2 if identical else 1)).simplify_full()
G_bb = width(["b", "b~"], (MB, MB))
G_bb
```

Out [62]:
$$-\frac{3(4e^2m_b^4 - e^2m_b^2M_H^2)\sqrt{-4m_b^2M_H^2 + M_H^4}}{32\pi M_H^3M_W^2s_W^2}$$

Minus the formula of part (a), $(\alpha m_h/(8\sin^2 \theta_w))(m_f^2/m_W^2)(1 - 4m_f^2/m_h^2)^{3/2}N_c$:

```
In [63]: al = var('al', latex_name=r'\alpha')
assume(MB > 0, MW > 0, MZ > 0, MH > 2*MB, MH > 2*MW, MH > 2*MZ)
peskin_a = al*MH/(8*SW^2) * MB^2/MW^2 * (1 - 4*MB^2/MH^2)^(3/2) * 3
(G_bb.subs(EL=sqrt(4*pi*al)) - peskin_a).canonicalize_radical().simplify_full()
```

Out [63]:
$$0$$

W^+W^- and Z^0Z^0 (with the factor 1/2 for two identical Z), minus Xianyu's (21.59) and (21.60):

```
In [64]: G_WW = width(["W+", "W-"], (MW, MW))
G_ZZ = width(["Z", "Z"], (MZ, MZ), identical=True)
tW, tZ = MH^2/MW^2, MH^2/MZ^2
x1559 = al*MH^3/(16*MW^2*SW^2) * (1 - 4/tW + 12/tW^2) * sqrt(1 - 4/tW)
x1560 = al*MH^3/(32*MW^2*SW^2) * (1 - 4/tZ + 12/tZ^2) * sqrt(1 - 4/tZ)
[(G.subs(EL=sqrt(4*pi*al)).subs(CW=MW/MZ) - x).canonicalize_radical().simplify_full() for G, x in ((G_WW, x1559), (G_ZZ, x1560))]
```

Out [64]:
$$[0, 0]$$

Part (b) also asks why, for $m_h \gg m_W$, $\Gamma(h \rightarrow W^+W^-) \approx \Gamma(h \rightarrow \varphi^+\varphi^-)$, the decay into the Goldstone bosons of the ungauged theory. The ratio at $m_h = 300, 1000$ and 5000 GeV goes to 1: at high energy the longitudinal W bosons are the Goldstone bosons that the Higgs mechanism ate.

```
In [65]: G_GG = width(["G+", "G-"], (0, 0))          # the Goldstone bosons of the ungauged theory
          (massless)
          [(G_WW/G_GG).subs(peskin).subs(MH=m).n(digits=6) for m in (300, 1000, 5000)]
```

```
Out[65]: [0.656623, 0.962332, 0.998465]
```

12.2 (c): $h \rightarrow gg$ at one loop

Now the diagrams have a loop. `insert_fields` with a one-loop topology gives the quark triangles (here b and t ; the lighter quarks give $I \rightarrow 0$). `loop_amplitude` builds each amplitude with a free index on each gluon, contracts it with the projector $T_{\mu\nu} = g_{\mu\nu} - (k_{2\mu}k_{1\nu} + k_{1\mu}k_{2\nu})/(k_1 \cdot k_2)$, traces the quark loop in d dimensions and reduces the integral to B_0 and C_0 . Gauge invariance gives $\mathcal{M}^{\mu\nu} = F(k_1 \cdot k_2 g^{\mu\nu} - k_2^\mu k_1^\nu)$, so $T_{\mu\nu} \mathcal{M}^{\mu\nu} = (d-2)(k_1 \cdot k_2)F$. The four diagrams and the pole of their sum, which must vanish:

```
In [66]: light = ['e', 'mu', 'ta', 'ne', 'nm', 'nt', 'u', 'c', 'd', 's']
dgg = insert_fields(topologies(1, 1, 2, exclude=("tadpoles", "wf")), ["H"], ["g", "g"], sm,
exclude_fields=light)
def transverse(num, ext):
    mu, nu = ext[1], ext[2]
    return num*(g(mu, nu) - (c_(k2, mu)*c_(k1, nu) + c_(k1, mu)*c_(k2, nu))/dot_(k1, k2))
kin = {"k1^2": 0, "k2^2": 0, "k1.k2": "MH^2/2"}
Mgg = sum(dgg.loop_amplitude([k1 + k2], [k1, k2], transverse, kin))
a2, a3 = color_indices("a2 a3")
F_gg = (finite_part(Mgg, 1) / MH^2).subs({color_delta(a2, a3): 1}) # F = T.M/((d - 2)
k1.k2); colour delta^{ab} stripped
len(dgg), uv_part(Mgg).simplify_full()
```

```
Out[66]: (4, 0)
```

The width against Peskin's formula in part (c) with the form factor of Xianyu's (21.67), at $m_h = 100, 300$ and 400 GeV (in GeV):

```
In [67]: import mpmath
def I_x(tau):
    # Xianyu (21.67), I = 3 int dx dy (1 - 4xy)/(1 - xy tau - i0), with the y integral done
    by hand:
    # 3 int_0^1 dx [ (4/tau)(1 - x) - (1 - 4/tau) log(1 - x(1 - x) tau - i0)/(x tau) ]
    tau = mpmath.mpf(float(tau))
    lg = lambda x: mpmath.log(mpmath.mpc(1 - x*(1 - x)*tau, -mpmath.mpf(10)**-40))
    pts = [0, 0.5, 1]
    if tau > 4:
        # the zeros of 1 - x(1 - x) tau, where
        the log is singular
        r = mpmath.sqrt(1 - 4/tau)/2
        pts = [0, 0.5 - r, 0.5, 0.5 + r, 1]
    return complex(3*mpmath.quad(lambda x: 4/tau*(1 - x) - (1 - 4/tau)*lg(x)/(x*tau), pts))
G_gg = abs(F_gg)^2 * MH^3/(8*pi) # sum |M|^2 = 8 colours x 2 (k1.k2)^2 |F|^2, Gamma
= sum |M|^2/(32 pi m_h)
def peskin_c(m):
    I = sum(I_x(float(m^2/mq^2)) for mq in (5, 175))
    return float((1/128)*m/(8*0.23)*m^2/80^2*0.12^2/(9*pi.n())^2*abs(complex(I))^2)
[(float(G_gg.subs(peskin).subs(MH=m).n()), peskin_c(m)) for m in (100, 300, 400)]
```

```
Out[67]: [(9.678636829959462 × 10-05, 9.678636829959887 × 10-05), (0.004450689608234522, 0.004450689608234508), (0.021881685449096615, 0.021881685449096605)]
```

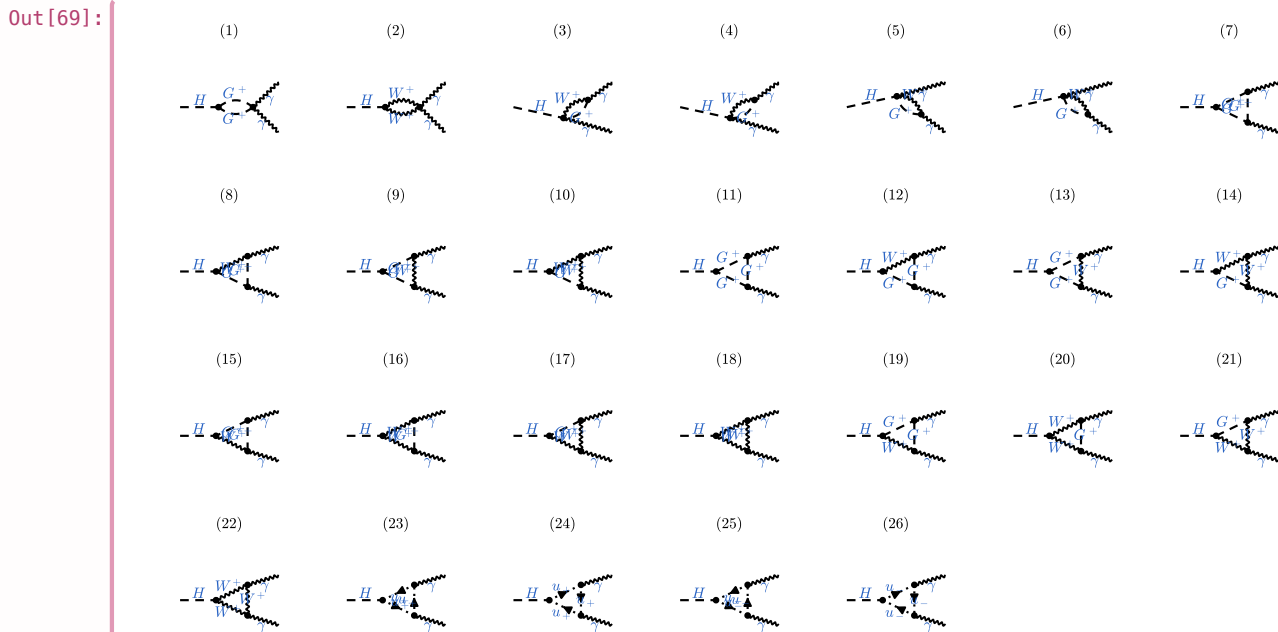
12.3 (e) and (f): $h \rightarrow \gamma\gamma$, quarks and the W loop

The photons couple to every charged particle. In Feynman-'t Hooft gauge there are the fermion triangles and 26 diagrams with W bosons, Goldstone bosons and ghosts, the "13 diagrams" of Xianyu counted with both orders of the photons. feynsage finds all of them (30 with the b and t quarks, the same count as FeynArts), and the poles cancel separately in the bosonic and the fermionic part:

```
In [68]: fermions_light = ['e', 'mu', 'ne', 'nm', 'nt', 'u', 'c', 'd', 's']
daa = insert_fields(topologies(1, 1, 2, exclude=("tadpoles", "wf")), ["H"], ["A", "A"], sm,
exclude_fields=fermions_light + ['ta'])
bosonic = DiagramList([x for x in daa if not any(sm.field(f).kind == "F" for f in
x.propagator_fields())], daa.tops, [])
fermionic = DiagramList([x for x in daa if any(sm.field(f).kind == "F" for f in
x.propagator_fields())], daa.tops, [])
kin = {"k1^2": 0, "k2^2": 0, "k1.k2": "MH^2/2"}
M_W = sum(bosonic.loop_amplitude([k1 + k2], [k1, k2], transverse, kin))
M_f = sum(fermionic.loop_amplitude([k1 + k2], [k1, k2], transverse, kin))
len(daa), len(bosonic), uv_part(M_W).simplify_full(), uv_part(M_f).simplify_full()
```

```
Out [68]: (30, 26, 0, 0)
```

```
In [69]: fig = bosonic.draw(ncols=7, size=1.25)
```



The W part in the normalisation of Xianyu's (21.96), $\mathcal{M}_W = (\alpha m_h^2 / (2\pi v)) I_W$, against his integral formula (21.97) below the WW threshold:

```
In [70]: def I_W(tau):
# Xianyu (21.97)-(21.100)
tau = mpmath.mpf(float(tau))
I1 = mpmath.quad(lambda x: mpmath.log(1 - x*(1 - x)*tau), [0, 1])
I2 = 2*mpmath.quad(lambda x: mpmath.quad(lambda y: mpmath.log(1 - x*y*tau), [0, 1 -
x]), [0, 1])
I3 = mpmath.quad(lambda x: mpmath.quad(lambda y: (8 - 3*x + y + 4*x*y)*tau/(1 -
x*y*tau), [0, 1 - x]), [0, 1])
return complex((6*I1 - 8*I2 + tau*(I1 - I2) + I3)/tau)
v_ev, alpha = 2*MW*SW/EL, EL^2/(4*pi)
R_W = (finite_part(M_W, 1)/MH^2) * pi*v_ev/alpha # = I_W if M_W = alpha m_h^2/(2 pi
v) I_W (below the W W threshold)
[(float(R_W.subs(peskin).subs(MH=m).n().real()), I_W(m^2/80^2).real) for m in (50, 100,
150)]
```

```
Out [70]: [(3.575286601917601, 3.5752866019176057), (3.859842117418596, 3.8598421174185953), (4.880311609088385, 4.880311609088385)]
```

For a light Higgs the W part goes to $7/2$ and the quark part (top and bottom) to $-(2/3)(4/3 + 1/3) = -10/9$:

```
In [71]: R_f = (finite_part(M_f, 1)/MH^2) * pi*v_ev/alpha
small = {MH: 1/10}
R_W.subs(peskin).subs(small).n(digits=40).real().n(digits=8),
R_f.subs(peskin).subs(small).n(digits=40).real().n(digits=8)
```

Out[71]: $(3.5000003, -1.1111163)$

Together the amplitude is $-(\alpha m_h^2/(3\pi v))[\sum Q_f^2 N_c I_f - 21/4]$, which is the formula of part (f) on p. 777. The full width, against Xianyu's (21.101) with the factor $3/2$ in front of I_W and as printed (without it), at $m_h = 60, 100$ and 150 GeV:

```
In [72]: F_aa = (finite_part(M_W + M_f, 1)/MH^2).subs(peskin)
def compare(m):
    G = float(abs(complex(F_aa.subs(MH=m).n()))^2 * m^3/(64*pi.n()))
    Sf = (4/9)*3*complex(I_x(m^2/175^2)) + (1/9)*3*complex(I_x(m^2/5^2))
    pref = (1/128)*m/(8*0.23)*m^2/80^2*(1/128)^2/(18*pi.n()^2)
    IW = I_W(m^2/80^2)
    return G, float(pref*abs(Sf - 1.5*IW)^2), float(pref*abs(Sf - IW)^2)
[compare(m) for m in (60, 100, 150)]
```

Out[72]: $[(8.3200892630383 \times 10^{-07}, 8.320089263030974 \times 10^{-07}, 2.6190905861479945 \times 10^{-07}), (4.528851540001814 \times 10^{-06}, 4.528851540001745 \times 10^{-06}, 1.4564067294020392 \times 10^{-06}), (2.7171456689050303 \times 10^{-05}, 2.7171456689050554 \times 10^{-05}, 9.439966911307296 \times 10^{-06})]$

A MISPRINT IN THE WORKED SOLUTION

Xianyu's (21.96) normalises the W loop so that $I_W \rightarrow 7/2$ for a light Higgs, and his (21.95) and Peskin's formula have $21/4$. Then the full result must be $|\sum Q_f^2 N_c I_f - (3/2)I_W|^2$. Eq. (21.101) prints $|\sum Q_f^2 N_c I_f - I_W|^2$. feynsage agrees with the version with $3/2$.

12.4 (d): Higgs production in pp collisions

With the gluon distribution $f_{g(x)} = 8(1-x)^7/x$ of the book, $I = 1$ and $\sigma(gg \rightarrow h) = (\pi^2/(8m_h))\Gamma(h \rightarrow gg)\delta(\hat{s} - m_h^2)$, the cross section for a 30 GeV Higgs at $\sqrt{s} = 1$ to 40 TeV, in nb:

```
In [73]: def sigma_pp_nb(rs, mh=30, I=1):
    # sigma(pp -> h + X) = int dx1 dx2 f_g(x1) f_g(x2) sigma(gg -> h), f_g(x) = 8 (1 - x)^7/x,
    # sigma(gg -> h) = pi^2/(8 m_h) Gamma(h -> gg) delta(s_hat - m_h^2) (Xianyu (21.71))
    G = float((1/128)*mh/(8*0.23)*mh^2/80^2*0.12^2/(9*pi.n()^2)*I^2)
    tau0 = mh^2/rs^2
    lum = mpmath.quad(lambda x1: (8*(1-x1)**7/x1)*(8*(1-tau0/x1)**7/(tau0/x1))/x1,
    [tau0, 1])
    return float(pi.n()^2/(8*mh)*G*lum/rs^2 * 0.3894e6) # GeV^-2 -> nb
roots = [1000*x for x in (1, 2, 5, 10, 20, 40)]
[(r/1000, round(sigma_pp_nb(r), 3)) for r in roots]
```

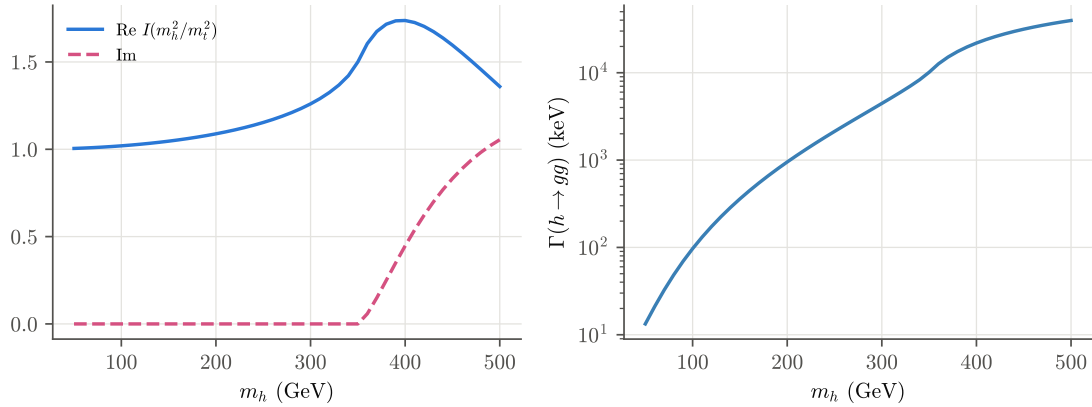
Out[73]: $[(1, 0.007), (2, 0.011), (5, 0.017), (10, 0.021), (20, 0.026), (40, 0.03)]$

12.5 (g) and (h): the form factor, $\Gamma(h \rightarrow gg)$ and the branching fractions

The form factor $I(m_h^2/m_t^2)$ (it becomes complex above $2m_t$) and $\Gamma(h \rightarrow gg)$ in keV:

```
In [74]: import numpy as np, matplotlib.pyplot as plt
mhs = np.linspace(50, 500, 46)
I_top = [complex(I_x(m**2/175**2)) for m in mhs]
G_gg_keV = [1e6*float(G_gg.subs(peskin).subs(MH=m).n()) for m in mhs]
fig, (a1, a2) = plt.subplots(1, 2, figsize=(7.4, 2.9))
a1.plot(mhs, [z.real for z in I_top], label=r"Re $I(m_h^2/m_t^2)$"); a1.plot(mhs, [z.imag for z in I_top], ls="--", label=r"Im")
a1.set_xlabel(r"$m_h$ (GeV)"); a1.legend(fontsize=8)
a2.semilogy(mhs, G_gg_keV, color="C3"); a2.set_xlabel(r"$m_h$ (GeV)"); a2.set_ylabel(r"$\Gamma(h \to gg)$ (keV)")
fig.tight_layout()
```

Out [74]:

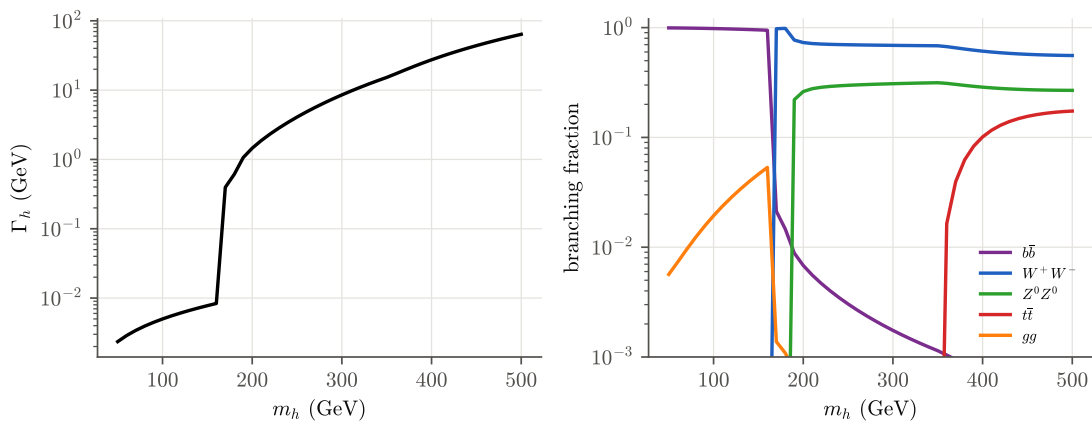


The total width and the branching fractions into $b\bar{b}$, $t\bar{t}$, gg , W^+W^- and Z^0Z^0 :

```
In [75]: Gs = {"bb": G_bb.subs(EL=sqrt(4*pi*a1)).subs(a1=1/128), "WW": G_WW, "ZZ": G_ZZ, "tt":
width(["t", "t~"], (MT, MT)), "gg": G_gg}
def partial(name, m):
    thr = {"WW": 2*80, "ZZ": 2*91, "tt": 2*175, "bb": 10, "gg": 0}[name]
    return 0.0 if m <= thr else float(abs(complex(Gs[name].subs(peskin).subs(MH=m).n()))))
table = {n: [partial(n, m) for m in mhs] for n in Gs}
total = [sum(table[n][i] for n in Gs) for i in range(len(mhs))]
fig, (a1, a2) = plt.subplots(1, 2, figsize=(7.4, 3.0))
a1.semilogy(mhs, total, color="k"); a1.set_xlabel(r"$m_h$ (GeV)"); a1.set_ylabel(r"$\Gamma_h$ (GeV)")
labels = {"bb": r"$b\bar{b}$", "tt": r"$t\bar{t}$", "gg": r"$gg$", "WW": r"$W^+W^-$", "ZZ": r"$Z^0Z^0$"}
colours = {"bb": "#7b2d8e", "tt": "#d62728", "gg": "#ff7f0e", "WW": "#1f5fbf", "ZZ": "#2ca02c"}
for n in Gs:
    a2.semilogy(mhs, [max(table[n][i]/total[i], 1e-6) for i in range(len(mhs))],
label=labels[n], color=colours[n])
print("BR(gg) at m_h = 100, 200, 300, 400 GeV:", [round(table["gg"][i]/total[i], 5) for i
in (5, 15, 25, 35)])
a2.set_ylim(1e-3, 1.2); a2.set_xlabel(r"$m_h$ (GeV)"); a2.set_ylabel("branching fraction");
a2.legend(fontsize=7)
fig.tight_layout()
```

BR(gg) at $m_h = 100, 200, 300, 400$ GeV: [0.01936, 0.00065, 0.00052, 0.0008]

Out [75]:



13 Two problems of Chapter 21: the muon $g - 2$ and $e^+e^- \rightarrow W^+W^-$

Problems 21.1 and 21.2 of Peskin and Schroeder (p. 773), checked against the book and against Xianyu's worked solutions (pp. 177 to 183 of the solutions).

13.1 Problem 21.1: weak contributions to the muon $g - 2$

The vertex $\bar{u}(p_2)X^\mu u(p_1)$ has the form $\gamma^\mu F_1 + (i\sigma^{\mu\nu}q_\nu/2m)F_2$ plus parity-odd pieces, and $a_\mu = F_2(0)$. `vertex_projector(p1, p2, m, mu, "F2")` gives the Dirac matrix P with $\text{Tr}[(\not{p}_1 + m)P(\not{p}_2 + m)X] = F_2$, built exactly in d dimensions. The parity-odd traces would need an ε tensor with four different vectors, but there are only p_1, p_2 and the loop momentum, so they vanish. `gamma5_scheme="NDR-even"` leaves them out. First the QED vertex, which must give Schwinger's $\alpha/(2\pi)$:

```
In [76]: from feynsage.models import MM
from feynsage import u as u_spinor
p1, p2 = momenta("p1 p2")          # muon in (p1), photon in (p2 - p1), muon
out (p2)
q2 = var("q2")                      # q^2 = (p2 - p1)^2, sent to 0 at the end
def F2_projector(num, ext):          # Tr[(p1/ + m) P (p2/ + m) X] picks F2 out
of X^mu
    P = vertex_projector(p1, p2, MM, ext[1], "F2")
    return dirac_trace((slash(p1) + MM)*P*(slash(p2) + MM)*num, dim="d",
gamma5_scheme="NDR-even")
vertex_kin = {"p1^2": "MM^2", "p2^2": "MM^2", "p1.p2": "MM^2 - q2/2"}
def at_q2_zero(F):
    return ((F.subs(q2=10^-10) + F.subs(q2=-10^-10))/2)
tops_v = topologies(1, 2, 1, exclude=("tadpoles", "wf"))
ALL = ["A", "Z", "W+", "H", "G0", "G+", "u+", "u-", "uA", "uZ", "g", "ug", "e", "mu", "ta",
"ne", "nm", "nt",
"u", "c", "t", "d", "s", "b"]
only = lambda keep: [f for f in ALL if f not in keep] # fields that may not run inside
the loop
dq = insert_fields(tops_v, ["mu", "A"], ["mu"], sm, exclude_fields=only(["A", "mu"]))
F2 = finite_part(sum(dq.loop_amplitude([p1, p2 - p1], [p2], F2_projector, kin=vertex_kin),
1)/EL
F2_qed = at_q2_zero(F2.subs({EL: 1, MM: 1})).n(digits=40)
F2_qed.n(digits=20), (1/(8*pi^2)).n(digits=20) # F2(0)/e^2 and alpha/(2 pi) with e = 1
```

```
Out [76]: (0.012665147955292221430, 0.012665147955292221430)
```

Now only $W^\pm$, $\varphi^\pm$ and ν_μ may run in the loop. The four diagrams of Xianyu's Fig. 21.1 in units of $G_F m_\mu^2 / (8\pi^2 \sqrt{2})$, for $m_W = 1000 m_\mu$:

```
In [77]: GF = EL^2/(4*sqrt(2)*SW^2*MM^2)          # G_F/sqrt(2) = g^2/(8 m_W^2), g = e/
sin(theta_w)
unit = GF*MM^2/(8*pi^2*sqrt(2))
heavy = {EL: 1, MM: 1, MW: 1000, SW: sqrt(23/100), CW: sqrt(77/100), MZ: 1000/sqrt(77/100)}
# m_W = 1000 m_mu
dW = insert_fields(tops_v, ["mu", "A"], ["mu"], sm, exclude_fields=only(["W+", "G+",
"nm"]))
def a_mu(diags, xi=1):
    res = diags.loop_amplitude([p1, p2 - p1], [p2], F2_projector, kin=vertex_kin, xi=xi)
    return [(at_q2_zero(finite_part(r, 1)/EL/unit).subs(heavy)).n(digits=60).n(digits=8)
for r in res]
[x.propagator_fields() for x in dW], a_mu(dW)
```

```
Out [77]: ([[nm, G+, G+], [nm, G+, W+], [nm, W+, G+], [nm, W+, W+]], [-3.3333350 x 10^-7, 0.50000017, 0.50000017, 2.3333340])
```

These are $7/3$ for the $WW\nu$ diagram, $1/2$ for each $W\varphi$ diagram and a negligible $\varphi\varphi$ diagram, as in Xianyu's (21.2), (21.5) and (21.7), and the sum is the $10/3$ of part (a). For part (b), in R_ξ gauge the W propagator is

split as $-i[g - kk/m_W^2]/(k^2 - m_W^2) - i(kk/m_W^2)/(k^2 - \xi m_W^2)$ and the Goldstone mass becomes $\sqrt{\xi}m_W$. The single diagrams change with ξ but the sum does not, from $\xi = 1/100$ to $\xi = 100$:

```
In [78]: [(xi, sum(a_mu(dW, xi))) for xi in (1, 3, 1/2, 1/100, 100)]
```

```
Out[78]: [(1, 3.3333340), (3, 3.3333340), (1/2, 3.3333340), (1/100, 3.3333340), (100, 3.3333340)]
```

Part (c), the Z and the neutral Goldstone boson, against $-(4/3 + (8/3)\sin^2\theta_w - (16/3)\sin^4\theta_w)$ of the book (the same as Xianyu's (21.9)):

```
In [79]: dZ = insert_fields(tops_v, ["mu", "A"], ["mu"], sm, exclude_fields=only(["Z", "G0", "mu"]))
sw2 = 23/100
[x.propagator_fields() for x in dZ], sum(a_mu(dZ)), -(4/3 + 8/3*sw2 -
16/3*sw2^2).n(digits=8)
```

```
Out[79]: ([[G0, mu, mu], [Z, mu, mu]], -1.6645238, -1.6645333)
```

Xianyu also writes each diagram as a Feynman-parameter integral before taking $m_W \gg m_\mu$: (21.2) for the $WW\nu$ diagram and (21.9) for the Z diagram. Evaluated with the full Δ at the real ratio $m_W/m_\mu = 760.7$, they agree with the diagrams of feynsage to 12 digits (left feynsage, right the integral):

```
In [84]: # Xianyu's Feynman-parameter forms at q^2 = 0 with the full Delta, at the real m_W/m_mu
import mpmath
mpmath.mp.dps = 30
r_real = (80379/1000)/(1056584/10^7)
real = {EL: 1, MM: 1, MW: r_real, SW: sqrt(sw2), CW: sqrt(1 - sw2), MZ: r_real/sqrt(1 -
sw2)}
def a_real(diags):
    res = diags.loop_amplitude([p1, p2 - p1], [p2], F2_projector, kin=vertex_kin)
    return [(at_q2_zero(finite_part(r, 1)/EL/unit).subs(real)).n(digits=40) for r in res]
def dbl(f):
    return mpmath.quad(lambda x: mpmath.quad(f, [0, 1 - x]), [0, 1])
R2, z2, c2 = mpmath.mpf(float(r_real))*2, mpmath.mpf(float(r_real))*2/(1 -
mpmath.mpf(0.23)), 1 - mpmath.mpf(0.23)
x21_2 = 2*R2*dbl(lambda y: (1 - y)*(3 - 2*y)/((1 - y)*R2 - y*(1 - y))) #
(21.2)
x21_9 = -(R2/(2*c2))*dbl(lambda y: (2*y*(3 + y) - (4*mpmath.mpf(0.23) - 1)*2*y*(1 - y))/
((1 - y)*2 + y*z2)) # (21.9)
[(a_real(dW)[3].n(digits=12), mpmath.nstr(x21_2, 12)), (a_real(dZ)[1].n(digits=12),
mpmath.nstr(x21_9, 12))]
```

```
Out[84]: [(2.33333448528, 2.33333448528), (-1.66448641094, -1.66448641094)]
```

13.2 Problem 21.2: $e^+e^- \rightarrow W^+W^-$ with longitudinal W bosons

`chiral(M, p1, "L")` keeps the left-handed electron, so `spin_sum` gives one helicity amplitude squared. A longitudinal polarization vector is a real combination of the two W momenta. $|\mathcal{M}|^2$ for $e_L^- e_R^+ \rightarrow W_0^+ W_0^-$ at $\sqrt{s} = 500$ GeV, $\cos\theta = -1/2$ (θ is the angle of the W^+ , as in Xianyu):

```

In [80]: from feynsage.models import ME
from feynsage.qft import tensors as T
from feynsage.diagrams import conjugate_amplitude
dww = insert_fields(topologies(0, 2, 2), ["e-", "e+"], ["W+", "W-"], sm)
q1, q2_, q3, q4 = momenta("q1 q2 q3 q4") # e-(q1) e+(q2) -> W+(q3) W-(q4)
Mww = dww.amplitude([q1, q2_], [q3, q4])
s_, t_, u_ = var("s t u")
kin_ww = {dot(q1,q1): 0, dot(q2_,q2_): 0, dot(q3,q3): MW^2, dot(q4,q4): MW^2, dot(q1,q2_):
s_/2,
          dot(q3,q4): (s_ - 2*MW^2)/2, dot(q1,q3): (MW^2 - t_)/2, dot(q2_,q4): (MW^2 -
t_)/2,
          dot(q1,q4): (MW^2 - u_)/2, dot(q2_,q3): (MW^2 - u_)/2}
# longitudinal polarization vectors, real: eps_L(q3) = [q3 (q3.q4) - q4 m_W^2]/(m_W
sqrt((q3.q4)^2 - m_W^4))
P34 = (s_ - 2*MW^2)/2; NL = MW*sqrt(P34^2 - MW^4)
eL3, eL4 = (P34*q3 - MW^2*q4)/NL, (P34*q4 - MW^2*q3)/NL
longitudinal = {T._declare(n, "vector"): v for n, v in (("eps3", eL3), ("eps3_c", eL3),
("eps4", eL4), ("eps4_c", eL4))}
def squared_00(hand): # |M(e-_hand e+ -> W+_0 W-_0)|^2
    Mh = chiral(Mww, q1, hand)
    return SR(spin_sum(Mh*conjugate_amplitude(Mh),
rules=longitudinal)).subs(kin_ww).subs({ME: 0})
LR, RL = squared_00("L"), squared_00("R")
mw_ = 804/10; sw2 = 23/100; mz_ = mw_/sqrt(1 - sw2)
num_ww = {EL: 1, SW: sqrt(sw2), CW: sqrt(1 - sw2), MW: mw_, MZ: mz_}
def pt_ww(rs, c): # theta = angle between e- and W+
    E = rs/2; p = sqrt(E^2 - mw_^2); tv = mw_^2 - rs^2/2 + 2*E*p*c
    return {s_: rs^2, t_: tv, u_: 2*mw_^2 - rs^2 - tv}
LR.subs(num_ww).subs(pt_ww(500, -1/2)).n(digits=20)

```

Out[80]: 1.16772149000000775499

$|\mathcal{M}|^2$ divided by the square of Xianyu's (21.24) in three forms: the first line, the second line as printed and the second line with the relative sign of its two terms changed, and the last column against (21.29) for $e_R^- e_L^+$:

```

In [81]: def X24_first(rs, c): # first line of Xianyu (21.24), without i
e^2
    sv = rs^2; E = rs/2; p = sqrt(E^2 - mw_^2); sn = sqrt(1 - c^2); uv = mw_^2 - sv/2 -
2*E*p*c
    return (mz_^2/(sv*(sv - mz_^2)) - 1/(2*sw2)/(sv - mz_^2))*(-4*E*p*(E^2 + p^2)/mw_^2 +
16*E^3*p/mw_^2)*sn + 1/(2*sw2)/uv*2*E*(-3*E^2*p + p^3 - 2*E^3*c)*sn/mw_^2
def X24_second(rs, c, sign): # second line of (21.24); printed with sign
= -1
    sv = rs^2; b = sqrt(1 - 4*mw_^2/sv); D = 1 + b^2 + 2*b*c
    return -sv/(4*mw_^2)*(mz_^2/(sv - mz_^2)*b*(3 - b^2) + sign/(2*sw2)*((2/D - sv/(sv -
mz_^2))*b*(3 - b^2) + 4*c/D))*sqrt(1 - c^2)
def X29(rs, c): # Xianyu (21.29)
    sv = rs^2; b = sqrt(1 - 4*mw_^2/sv)
    return sv/(sv - mz_^2)*mz_^2/(4*mw_^2)*b*(b^2 - 3)*sqrt(1 - c^2)
rows = []
for rs, c in [(200, 3/10), (500, -1/2), (1000, 4/5)]:
    a, r = LR.subs(num_ww).subs(pt_ww(rs, c)), RL.subs(num_ww).subs(pt_ww(rs, c))
    rows.append([rs, c] + [(a/x^2).n(digits=60).n(digits=15) for x in (X24_first(rs, c),
X24_second(rs, c, -1), X24_second(rs, c, +1))]
+ [(r/X29(rs, c)^2).n(digits=60).n(digits=15)])
print("sqrt(s) cos first line printed 2nd 2nd with + (21.29)")
for row in rows:
    print("%-8s %-5s " % (row[0], row[1]) + " ".join("%-14.10f" % float(x) for x in
row[2:]))

```

sqrt(s)	cos	first line	printed 2nd	2nd with +	(21.29)
200	3/10	1.0000000000	4.3044487432	1.0000000000	1.0000000000
500	-1/2	1.0000000000	190.2837164428	1.0000000000	1.0000000000
1000	4/5	1.0000000000	95.1490761247	1.0000000000	1.0000000000

The first line of (21.24) and (21.29) agree. The second line of (21.24) does not. The same amplitude against the book's own complete expression (21.108) (p. 757, where θ is the angle of the W^-):

```
In [82]: J = chiral(vbar(q2_)*slash(q3 - q4)*u_spinor(q1), q1, "L") # vbar_L gamma_L u_L (k+ - k-)^L
J2 = SR(spinn_sum(J*conjugate_amplitude(J))).subs(kin_ww)
def P108(rs, cP): # the bracket of Peskin (21.108); theta_P = angle of W-
    sv = rs^2; b = sqrt(1 - 4*mw_^2/sv)
    return (1/(2*sw2)*(-sv/(sv - mz_^2)*(mz_^2/(2*mw_^2) + 1) + 2/b^2 - 8*mw_^2/(sv*b^2*(1 + b^2 - 2*b*cP)))
            + mz_^2/mw_^2*((sv/2 + mw_^2)/(sv - mz_^2)))/sv
    [(rs, c, (LR.subs(num_ww).subs(pt_ww(rs, c)))/(J2.subs(num_ww).subs(pt_ww(rs, c))*P108(rs, -c)^2)).n(digits=60).n(digits=15))
    for rs, c in [(200, 3/10), (500, -1/2), (1000, 4/5)]]
```

```
Out[82]: [ (200, 3/10, 1.0000000000000000), (500, -1/2, 1.0000000000000000), (1000, 4/5, 1.0000000000000000) ]
```

At high energy the longitudinal W bosons are the Goldstone bosons. feynsage makes the diagrams for $e_L^- e_R^+ \rightarrow \varphi^+ \varphi^-$ directly. The columns are $|\mathcal{M}(W_0 W_0)|^2/|\mathcal{M}(\varphi^+ \varphi^-)|^2$, the same against $(e^2 \sin \theta / (4s_w^2 c_w^2))^2$ and against the square of Xianyu's (21.28), at $\cos \theta = 0.3$ and $\sqrt{s} = 1$ to 1000 TeV:

```
In [83]: dgg = insert_fields(topologies(0, 2, 2), ["e-", "e+"], ["G+", "G-"], sm)
Mg = chiral(dgg.amplitude([q1, q2_], [q3, q4]), q1, "L")
GG = SR(spinn_sum(Mg*conjugate_amplitude(Mg))).subs(kin_ww).subs({ME: 0})
c = 3/10
def r(x, y, rs):
    return (x.subs(num_ww).subs(pt_ww(rs, c)).n(digits=80)/y.subs(num_ww).subs(pt_ww(rs, c)).n(digits=80)).n(digits=8)
lim = (1 - c^2)/(4*sw2*(1 - sw2))^2 # (e^2 sin(theta)/(4 s_w^2 c_w^2))^2
X28 = (1 + 2*c)^2*(1 - c^2)/(2*sw2*(1 + c))^2 # Xianyu (21.28) squared
print("sqrt(s)/GeV   W0W0/phi phi   W0W0/limit   W0W0/(21.28)")
for rs in (10^3, 10^4, 10^5, 10^6):
    x = LR.subs(num_ww).subs(pt_ww(rs, c)).n(digits=80)
    print("%-13d %-14.8f %-12.8f %-8.8f" % (rs, float(r(LR, GG, rs)), float(x/lim), float(x/X28)))
```

sqrt(s)/GeV	W0W0/phi phi	W0W0/limit	W0W0/(21.28)
1000	1.07273175	1.05016046	0.29232164
10000	1.00070117	1.00049141	0.27849581
100000	1.00000701	1.00000491	0.27836039
1000000	1.00000007	1.00000005	0.27835903

13.3 All helicities, and Fig. 21.10 of the book

process() (Chapter 11) gives any helicity state directly. The electrons are made massless and the W helicities are +1, -1 and 0, with the polarization vectors of Jacob and Wick. They are the vectors of Xianyu's (21.12):

```
In [85]: # every helicity state with process(): electrons massless, the W helicities +1, -1, 0
          (Jacob-Wick vectors)
W = process("e- e+ -> W+ W-", massless=["e"])
num_W = {EL: 1, SW: sqrt(sw2), CW: sqrt(1 - sw2), MW: mw_, MZ: mz_} # e = 1, as above
def hel(he, h3, h4, rs, c, diagrams=None): # he = helicity of the e- (the e+ has the opposite)
    r = W.squared(helicities={"e-": he, "e+": -he, "W+": h3, "W-": h4}, diagrams=diagrams)
    return r.subs(num_W).subs({W.sqrt_s: rs, W.cos_theta: c}).n(digits=60)
hel(-1, 0, 0, 500, -1/2) # the same number as Out above for e-
                           _L e+_R -> W0 W0
```

```
Out[85]: 1.16772149000007754432962891523992860395271506655960068015511
```

Every amplitude of (21.25) to (21.32), at $\sqrt{s} = 500$ GeV and $\cos \theta = -1/2$. The columns are feynsage's $|\mathcal{M}|^2$ divided by the square of the first and of the second line of Xianyu's formula:

```
In [86]: def X(name, rs, c, pm): # Xianyu's printed amplitudes without i e^2 (first and
second lines)
    sv = rs^2; E0 = rs/2; p = sqrt(E0^2 - mw_^2); b = p/E0; sth = sqrt(1 - c^2); D = 1 +
b^2 + 2*b*c
    uv = mw_^2 - 2*E0^2 - 2*E0*p*c
    br = mz_^2/(sv*(sv - mz_^2)) - 1/(2*sw2)/(sv - mz_^2)
    return {"25a": br*(8*E0^2*p/mw_)*(-pm + c)/sqrt(2) - 1/(2*sw2)/uv*(2*E0/mw_)*(E0^2*(2*c
- pm) + 2*E0*p + pm*p^2)*(pm + c)/sqrt(2),
    "25b": (mz_^2/(sv - mz_^2)*b - 1/(2*sw2)*(sv/(sv - mz_^2)*b + (pm - 2*c - 2*b -
pm*b^2)/D))*(rs/mw_)*(pm + c)/sqrt(2),
    "26a": br*(-4*E0*p*sth) + 1/(2*sw2)/uv*2*E0*(p + E0*c)*sth,
    "26b": (-mz_^2/(sv - mz_^2)*b + 1/(2*sw2)*(sv/(sv - mz_^2)*b - 2*(b + c)/
D))*sth,
    "27a": -1/(2*sw2)/uv*2*E0^2*(-pm + c)*sth, "27b": 1/(2*sw2)*2*(pm - c)*sth/D,
    "30": mz_^2/(sv - mz_^2)*(rs/mw_)*b*(pm - c)/sqrt(2), "31": mz_^2/(sv -
mz_^2)*b*sth}[name]
    rs, c = 500, -1/2
    print("e- W+ W- eq. feynsage |M|^2 / Xianyu^2: first line second line")
    for pm in (1, -1):
        for hand, h3, h4, eq in (("L", 0, pm, "25"), ("L", -pm, 0, "25"), ("L", pm, pm, "26"),
("L", pm, -pm, "27"),
("R", 0, pm, "30"), ("R", -pm, 0, "30"), ("R", pm, pm, "31"),
("R", pm, -pm, "32")):
            v = hel({"L": -1, "R": +1}[hand], h3, h4, rs, c)
            if eq == "32":
                print("%s %d %d (21.32) |M|^2 = %.1e" % (hand, h3, h4, float(v)))
                continue
            cols = [X(eq + "a", rs, c, pm), X(eq + "b", rs, c, pm)] if eq in ("25", "26", "27")
            else [X(eq, rs, c, pm)]
            print("%s %d %d (21.%s) " % (hand, h3, h4, eq) + " ".join("%.12f" %
float(v/x^2) for x in cols))
```

```
e- W+ W- eq. feynsage |M|^2 / Xianyu^2: first line second line
L +0 +1 (21.25) 0.001263677247 1.000000000000
L -1 +0 (21.25) 0.001263677247 1.000000000000
L +1 +1 (21.26) 1.000000000000 1.000000000000
L +1 -1 (21.27) 1.000000000000 1.000000000000
R +0 +1 (21.30) 1.000000000000
R -1 +0 (21.30) 1.000000000000
R +1 +1 (21.31) 1.000000000000
R +1 -1 (21.32) |M|^2 = 0.0e+00
L +0 -1 (21.25) 0.003872720425 1.000000000000
L +1 +0 (21.25) 0.003872720425 1.000000000000
L -1 -1 (21.26) 1.000000000000 1.000000000000
L -1 +1 (21.27) 1.000000000000 1.000000000000
R +0 -1 (21.30) 1.000000000000
R +1 +0 (21.30) 1.000000000000
R -1 -1 (21.31) 1.000000000000
R -1 +1 (21.32) |M|^2 = 0.0e+00
```

Everything agrees except the first line of (21.25). The same numbers also come out of an explicit calculation with 4×4 Dirac matrices and spinors and no traces: all 54 values (18 helicity states at three energies) agree to 55 digits.

The misprints do not depend on feynsage at all. The next cell uses only the printed formulas and asks whether each line equals the next one. A is the m_Z^2 term inside the braces of (21.24) and B the $1/(2s_w^2)$ term:

```
In [87]: # only the printed formulas: does each line equal the next one? (no feynsage, no diagrams)
Ev, pv, cv, wv = var("E p c w")
mwv = sqrt(Ev^2 - pv^2); mzv = mwv^2/(1 - wv); sv = 4*Ev^2; bv = pv/Ev; snv = sqrt(1 - cv^2); Dv = 1 + bv^2 + 2*bv*cv
uv = mwv^2 - 2*Ev^2 - 2*Ev*pv*cv
brv = mzv/(sv*(sv - mzv)) - 1/(2*wv)/(sv - mzv)
first24 = brv*(-4*Ev*pv*(Ev^2 + pv^2)/mwv^2 + 16*Ev^3*pv/mwv^2)*snv + 1/(2*wv)/uv*2*Ev*(-3*Ev^2*pv + pv^3 - 2*Ev^3*cv)*snv/mwv^2
A = mzv/(sv - mzv)*bv*(3 - bv^2); B = 1/(2*wv)*((2/Dv - sv/(sv - mzv))*bv*(3 - bv^2) + 4*cv/Dv)
second24 = lambda sA, sB: -sv/(4*mwv^2)*(sA*A + sB*B)*snv
zero = lambda x: bool(x.simplify_full() == 0)
first27 = lambda pm: -1/(2*wv)/uv*2*Ev^2*(-pm + cv)*snv
second27 = lambda pm: 1/(2*wv)*2*(pm - cv)*snv/Dv
tt = var("tt") # m_W/E -> 0
print("(21.24) first line = second line as printed, {A - B}:", zero(first24 - second24(1, -1)))
print("(21.24) first line = second line with {-A - B}: ", zero(first24 - second24(-1, -1)))
print("(21.27) first = second:", zero(first27(1) - second27(1)), " first = -(second):", zero(first27(1) + second27(1)))
lim = (first24/snv).subs({pv: Ev*sqrt(1 - tt^2)}).simplify_full().limit(tt=0).factor()
print("high-energy limit of the (21.24) first line, divided by sin(theta):", lim)
print("Peskin (21.107) bracket 1/(2c^2) - 1/(4c^2 s^2) + 1/(2s^2):", (1/(2*(1 - wv)) - 1/(4*(1 - wv)*wv) + 1/(2*wv)).factor())

(21.24) first line = second line as printed, {A - B}: False
(21.24) first line = second line with {-A - B}: True
(21.27) first = second: False first = -(second): True
high-energy limit of the (21.24) first line, divided by sin(theta): -1/4/((w - 1)*w)
Peskin (21.107) bracket 1/(2c^2) - 1/(4c^2 s^2) + 1/(2s^2): -1/4/((w - 1)*w)
```

MISPRINTS IN THE WORKED SOLUTION OF PROBLEM 21.2

All checked on the rendered pages of Xianyu's solutions (pp. 182 and 183) and by algebra with the printed formulas only.

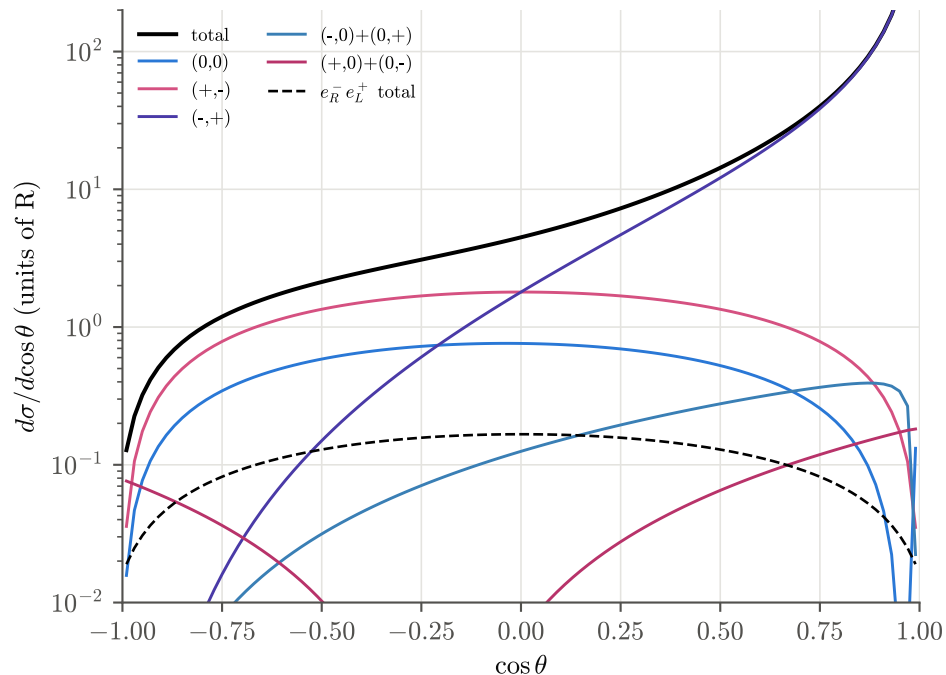
- (21.24): the second line has $-ie^2(s/4m_W^2)\{A - B\}$. It equals the first line only with $\{-A - B\}$, so the sign of the m_W^2 term is wrong. Peskin's (21.108) agrees with the first line.
- (21.25): in the first line the factor $(\mp 1 + \cos \theta)/\sqrt{2}$ should be $(\pm 1 + \cos \theta)/\sqrt{2}$. The second line, the final result, is right.
- (21.27): the two sides of the equation differ by an overall sign. This does not change $|\mathcal{M}|^2$.
- (21.28): the high-energy limit of the first line of (21.24) is $e^2 \sin \theta / (4s_w^2 c_w^2)$ with no other θ dependence. This is also Peskin's (21.107), where $1/(2c_w^2) - 1/(4c_w^2 s_w^2) + 1/(2s_w^2) = 1/(4s_w^2 c_w^2)$, and it is what the equivalence theorem gives. The printed $(1 + 2 \cos \theta)/(1 + \cos \theta)$ is not right.

Part (b) of the problem asks for Fig. 21.10 of the book (p. 757): $d\sigma/d\cos\theta$ for $e_L^- e_R^+ \rightarrow W^- W^+$ at $E_{\text{cm}} = 1000$ GeV in units of $R = 4\pi\alpha^2/(3E_{\text{cm}}^2)$ (Eq. (5.15), p. 140), with θ the angle of the W^- and the curves labelled by the helicities (h_-, h_+) . The dashed line is $e_R^- e_L^+$:

```
In [88]: import numpy as np, matplotlib.pyplot as plt
# Peskin Fig. 21.10: eL- eR+ at Ecm = 1000 GeV, theta = angle of the W- (cos = -
cos_theta of the W+),
# curves (h-, h+); dsigma/dcos in units of R = 4 pi alpha^2/(3 E_cm^2) is (3/8) beta |M|^2
with e = 1
rs = 1000; beta = sqrt(1 - 4*mw_^2/rs^2).n()
pairs = [(hm, hp) for hm in (1, -1, 0) for hp in (1, -1, 0)]
cfg = [{"e-": he, "e+": -he, "W+": hp, "W-": hm} for he in (-1, 1) for hm, hp in pairs]
tab = W.helicity_table(cfg) # all 18 states, the traces in
parallel
f = [fast_callable(SR(x.subs(num_W).subs({W.sqrt_s: rs})), vars=[W.cos_theta], domain=CDF)
for x in tab]
cPs = [k/100 for k in range(-99, 100, 2)]
ds = lambda k, cP: 3/8*float(beta)*f[k](-cP).real()
cur = {pq: [ds(k, x) for x in cPs] for k, pq in enumerate(pairs)}
tot = [sum(cur[pq][i] for pq in pairs) for i in range(len(cPs))]
totR = [sum(ds(9 + k, x) for k in range(9)) for x in cPs]
fig, ax = plt.subplots(figsize=(5.2, 3.8))
ax.semilogy(cPs, tot, "k", lw=1.5, label="total")
for pq, lab in (((0, 0), "(0,0)"), ((1, -1), "(+,-)"), ((-1, 1), "(-,+)")):
    ax.semilogy(cPs, cur[pq], lw=1.1, label=lab)
ax.semilogy(cPs, [a + b for a, b in zip(cur[(-1, 0)], cur[(0, 1)])], lw=1.1,
label="(-,0)+(0,+)")
ax.semilogy(cPs, [a + b for a, b in zip(cur[(1, 0)], cur[(0, -1)])], lw=1.1,
label="(+,0)+(0,-)")
ax.semilogy(cPs, totR, "k--", lw=1, label=r"$e^-_R e^+_L$ total")
ax.set_ylim(0.01, 200); ax.set_xlim(-1, 1); ax.set_xlabel(r"$\cos\theta$");
ax.set_ylabel(r"$d\sigma/d\cos\theta$ (units of R)")
ax.legend(fontsize=7, ncol=2); fig.tight_layout()
i0 = cPs.index(1/100)
print("eL- / eR+ at cos = 0.01:", round(tot[i0]/totR[i0], 1), " (+,+) and (-,-) largest
for |cos| < 0.9:",
"%1e" % max(max(cur[(1, 1)][i], cur[(-1, -1)][i]) for i in range(len(cPs)) if
abs(cPs[i]) < 0.9))
```

e_L⁻ / e_R⁺ at cos = 0.01: 27.3 (+,+) and (-,-) largest for |cos| < 0.9: 3.6e-03

Out[88]:



The curves have the same shapes and sizes as the book's figure. Near $\cos\theta = 0$ the e_R^- cross section is 27 times smaller, the "about 30 times" of the problem, and the (+,+) and (-,-) states stay below 0.01 R, too small to be seen in the figure. The last claim of part (a): when one W is longitudinal and the other transverse, each

diagram grows like $\sqrt{s}$ but the sum falls like $1/\sqrt{s}$. In $|\mathcal{M}|^2$ the single diagrams grow like s and the sum falls like $1/s$:

```
In [89]: # one W longitudinal, one transverse (21.2a): single diagrams grow like sqrt(s), the sum
         falls like 1/sqrt(s)
         names = [str(x.propagator_fields()[0]) for x in W.diagrams]
         print("|M|^2 for e-_L e+_R -> W+_0 W-_(+), cos = 0.3; diagrams:", names)
         for rs in (10^3, 10^4, 10^5):
             print("sqrt(s) = %6d GeV: " % rs + " ".join("%9.3e" % float(hel(-1, 0, 1, rs, 3/10,
             diagrams=[k])) for k in range(1, 5))
                 + " sum: %9.3e" % float(hel(-1, 0, 1, rs, 3/10)))

|M|^2 for e-_L e+_R -> W+_0 W-_(+), cos = 0.3; diagrams: ['A', 'H', 'Z', 'ne']
sqrt(s) = 1000 GeV: 1.273e+02 0.000e+00 1.785e+02 6.090e+02 sum: 1.214e-03
sqrt(s) = 10000 GeV: 1.307e+04 0.000e+00 1.801e+04 6.177e+04 sum: 1.269e-05
sqrt(s) = 100000 GeV: 1.307e+06 0.000e+00 1.801e+06 6.178e+06 sum: 1.270e-07
```

14 Where to go next

WHAT FEYNSAGE IS FOR

feynsage is meant for small and medium problems and for studying. Every step is exact, can be checked and explains itself. For very large reductions Kira and FIRE remain the better choice. I may continue improving feynsage in the future.

- github.com/aburousan/feynsage: the code and the latest version.
- [examples/feynsage_walkthrough.ipynb](#): a notebook that goes through the whole package.
- My notes of the lecture, *Feynman integrals at one loop*, with every derivation in full. They are not finished yet and will be shared once complete.
- References used in the lecture
 - P. Ramond, *Field Theory: A Modern Primer* (2nd ed.), Sections 4.3 and 4.4 and Appendix B
 - M. Argeri and P. Mastrolia, *Feynman diagrams and differential equations*, Int. J. Mod. Phys. A 22 (2007) 4375 (arXiv:0707.4037)
 - V. A. Smirnov, *Feynman Integral Calculus* (Springer 2006), chapter 3
 - S. Weinzierl, *Feynman Integrals* (Springer 2022, arXiv:2201.03593), Section 2.5 and Chapter 3
 - T. Rauh, *Introduction to Feynman integrals and multiloop techniques* (lecture notes, 2019). They follow Smirnov's later book, *Analytic Tools for Feynman Integrals* (Springer 2012)
- Sir's lectures on the same topic at IMSc, Chennai: [Introductory lectures on Feynman Integrals Part1](#) (video, IMSc YouTube channel)
- For the pion
 - S. L. Adler, *Axial-vector vertex in spinor electrodynamics*, Phys. Rev. 177 (1969) 2426